

NIYAMA: Breaking the Silos of LLM Inference Serving

Kanishk Goel, Jayashree Mohan, Nipun Kwatra, Ravi Shreyas Anupindi, Ramachandran Ramjee

Microsoft Research India

Abstract

The widespread adoption of Large Language Models (LLMs) has enabled diverse applications with very different latency requirements. Existing LLM serving frameworks rely on siloed infrastructure with coarse-grained workload segregation — interactive and batch — leading to inefficient resource utilization and limited support for fine-grained Quality-of-Service (QoS) differentiation. This results in operational inefficiencies, over-provisioning and poor load management during traffic surges.

We present NIYAMA, a novel QoS-driven inference serving system that enables efficient co-scheduling of diverse workloads on shared infrastructure. NIYAMA introduces fine-grained QoS classification allowing applications to specify precise latency requirements, and dynamically adapts scheduling decisions based on real-time system state. Leveraging the predictable execution characteristics of LLM inference, NIYAMA implements a dynamic chunking mechanism to improve overall throughput while maintaining strict QoS guarantees. Additionally, NIYAMA employs a hybrid prioritization policy that balances fairness and efficiency, and employs selective request relegation that enables graceful service degradation during overload conditions. Our evaluation demonstrates that NIYAMA increases serving capacity by 32% compared to current siloed deployments, while maintaining QoS guarantees. Notably, under extreme load, our system reduces SLO violations by an order of magnitude compared to current strategies.

1 Introduction

Large language models (LLMs) have transformed applications across diverse domains including conversational assistants, coding assistants, content generation, and summarization. These applications can have very different latency requirements — for example, autocomplete coding assistants demand responses within milliseconds, while summarization tasks can reasonably tolerate longer latencies. As LLM deployments scale to serve billions of users and diverse applications, inference serving systems must efficiently handle this diverse spectrum of latency requirements while ensuring high GPU utilization.

Current LLM serving solutions primarily adopt a coarse-grained categorization, segregating requests into two broad service classes: latency-sensitive interactive applications, and throughput-oriented batch processing, and serving them

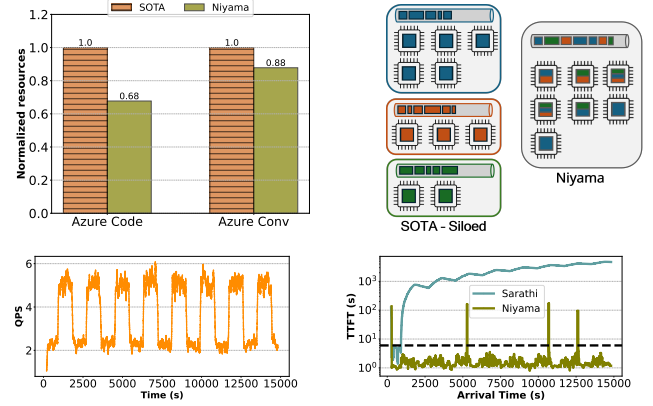


Figure 1. Efficiency of NIYAMA under uniform load and transient overload conditions. (top left) Normalized GPUs needed to serve a fixed traffic load while meeting the QoS targets of requests divided equally among 3 QoS tiers. Across two datasets, NIYAMA improves efficiency by 12-32% compared to the State-of-the-art (SOTA) Sarathi-Serve [4] siloed deployments. (top right) Illustration of NIYAMA co-scheduling vs current siloed deployments. (bottom left) Bursty overload scenario. (bottom right) NIYAMA maintains low latency while SOTA scheduling succumbs to cascading deadline violations under such bursty loads.

independently [13]. Interactive requests are typically served with smaller prefill chunks [4] to minimize latency, but that can result in relatively higher operational costs due to reduced throughput (e.g., 28% lower as shown in Figure 4). Batch requests, on the other hand, employ larger chunks to achieve higher throughput as latency is not a constraint. This siloed deployment, however, creates other inefficiencies: it leads to significant GPU resource under-utilization, as workload demands fluctuate across the two classes. Moreover, such partitioning inhibits the introduction of more QoS classes with fine-grained latency requirements, as doing so further exacerbates the partitioning inefficiencies.

Furthermore, current inference systems struggle under load fluctuations and overload conditions. Typical scheduling mechanisms such as first-come-first-served (FCFS) indiscriminately delay all incoming requests under overload, degrading user experience across the board. Alternatively, simplistic throttling approaches reject all new incoming requests when reaching capacity, ignoring their QoS requirements or relative priorities. Neither strategy adequately manages

5
2
0
2
r
a
M
8
2
J
G
L
s
C
1
v
2
6
5
2
2
3
0
5
2
v
i
X
r
a

Niyama: 打破 LLM 推理服务的孤岛

Kanishk Goel、Jayashree Mohan、Nipun Kwatra、Ravi Shreyas Anupindi、Ramachandran Ramjee 微软印度研究院

抽象的

大型语言模型 (LLM) 的广泛采用使得各种应用程序具有截然不同的延迟要求。现有的 LLM 服务框架依赖于具有粗粒度工作负载隔离（交互式和批量）的孤立基础设施，导致资源利用效率低下，并且对细粒度服务质量（QoS）差异化的支持有限。这会导致流量激增期间运营效率低下、过度配置和负载管理不佳。

我们推出了 Niyama，一种新颖的 QoS 驱动的推理服务系统，可以在共享基础设施上高效地共同调度不同的工作负载。Niyama 引入了细粒度的 QoS 分类，允许应用程序指定精确的延迟要求，并根据实时系统状态动态调整调度决策。利用 LLM 推理的可预测执行特性，Niyama 实现了动态分块机制，以提高整体吞吐量，同时保持严格的 QoS 保证。此外，Niyama 采用混合优先级策略来平衡公平性和效率，并采用选择性请求降级，从而在过载情况下实现平稳的服务降级。我们的评估表明，与当前的孤立部署相比，Niyama 将服务容量提高了 32%，同时保持了 QoS 保证。值得注意的是，在极端负载下，与当前策略相比，我们的系统将 SLO 违规减少了一个数量级。

1 简介

大型语言模型 (LLM) 已经改变了不同领域的应用程序，包括对话助手、编码助手、内容生成和摘要。这些应用程序可能有非常不同的延迟要求 - 例如，自动完成编码助手要求在几毫秒内响应，而摘要任务可以合理地容忍更长的延迟。随着 LLM 部署规模不断扩大，以服务数十亿用户和多样化的应用程序，推理服务系统必须有效地处理各种延迟要求，同时确保 GPU 的高利用率。

当前的 LLM 服务解决方案主要采用粗粒度分类，将请求分为两大类服务：延迟敏感的交互式应用程序和面向吞吐量的批处理，并为它们提供服务

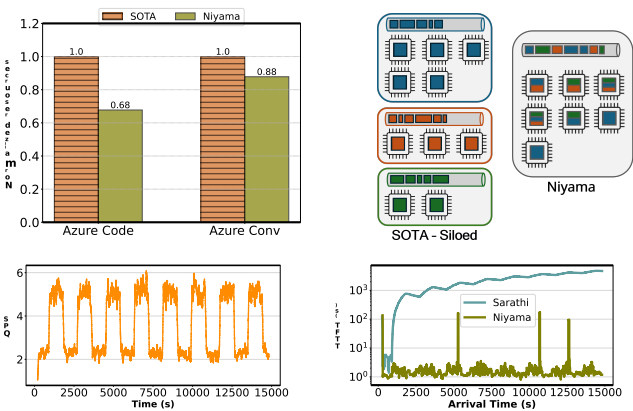


图 1. Niyama 在均匀负载和瞬态过载条件下的效率。（左上）标准化 GPU 需要服务固定流量负载，同时满足在 3 个 QoS 层之间平均分配的请求的 QoS 目标。在两个数据集上，与最先进的 (SOTA) Sarathi-Serve [4] 孤立部署相比，Niyama 将效率提高了 12-32%。（右上）Niyama 联合调度与当前孤立部署的图示。（左下）突发过载场景。（右下）Niyama 保持低延迟，而 SOTA 调度在这种突发负载下会屈服于级联的最后期限违规。

独立地[13]。交互式请求通常使用较小的预填充块 [4] 来服务，以最大程度地减少延迟，但这可能会由于吞吐量降低而导致相对较高的运营成本（例如，如图 4 所示，降低 28%）。另一方面，批量请求使用更大的块来实现更高的吞吐量，因为延迟不是限制。然而，这种孤立的部署会造成其他效率低下的问题：由于工作负载需求在两个类别之间波动，因此会导致 GPU 资源利用率显著不足。此外，这种划分会抑制引入更多具有细粒度延迟要求的 QoS 类别，因为这样做会进一步加剧划分效率低下的情况。

此外，当前的推理系统在负载波动和过载条件下难以应对。典型的调度机制，例如先来先服务 (FCFS)，会在过载情况下不加区别地延迟所有传入请求，从而全面降低用户体验。或者，简单的限制方法在达到容量时拒绝所有新传入的请求，忽略它们的 QoS 要求或相对优先级。这两种策略都没有充分管理

the complex trade-offs between throughput, latency, and fairness during such demand surges.

In this paper, we present NIYAMA, a QoS-driven LLM inference serving system that addresses these limitations through two key ideas. First, NIYAMA supports fine-grained QoS classes which allows applications to precisely specify their latency requirements. Multiple QoS classes are served efficiently by *co-scheduling requests with diverse QoS targets on a shared rather than siloed infrastructure*. Second, NIYAMA implements a hybrid prioritization and an eager relegation policy that *allows graceful service degradation during overload conditions*. Figure 1 compares NIYAMA to state-of-the-art Sarathi-Serve [4] siloed deployment, demonstrating significant performance improvements.

Efficiently supporting multiple QoS classes on a shared serving instance poses significant challenges. One approach is to use the smallest chunk size necessary to meet the latency constraint of the strictest QoS class on all serving instances. However, this would result in low throughput [4] and high cost for all service classes. Instead, NIYAMA leverages the unique execution characteristics of LLM inference – particularly the distinct prefill and decode phases and the inherent predictability of the prefill phase – to dynamically adjust chunk sizes based on the observed system state and individual QoS targets. Co-serving multiple QoS classes allows us to exploit *deadline slack* of requests with relaxed latency requirements to schedule bursts of larger chunk sizes, thereby increasing throughput opportunistically. Our evaluations demonstrate that NIYAMA achieves up to 32% higher effective serving capacity compared to current siloed deployments, while consistently meeting QoS guarantees across all request categories.

For managing overload conditions gracefully, NIYAMA employs a hybrid prioritization and an eager relegation policy. Simple overload handling approaches like shortest-job-first (SJF) manage overload by prioritizing short requests. This helps reduce load due to the quadratic dependence of request length on LLM system load [22]. However, SJF neglects the QoS requirements of longer jobs, leading to SLO violations even at low load (Figure 2). On the other hand, Earlier Deadline First (EDF) scheduling is optimal under low load but suffers excessive violations even when load is slightly higher than capacity. Thus, NIYAMA introduces a hybrid policy that smoothly interpolates between EDF and SJF, allowing deployments to minimize SLO violations across both low and high load. Additionally, NIYAMA proactively employs *eager relegation*, selectively degrading service for a small subset of requests to ensure stable performance, even under extreme load conditions. In multi-QoS scenarios, NIYAMA leverages application-provided hints about request importance, such as whether a request originates from a free or paid tier, to perform relegation. This ensures that lower-priority requests are affected first during overload conditions, allowing the system to maintain QoS for the majority of high-priority

requests. Our evaluations show that during significant overload scenarios (50% above capacity), NIYAMA consistently meets latency targets for over 95% of requests, translating into substantial cost savings and enhanced user experience across diverse applications relying on LLM infrastructure.

Our work makes the following key contributions:

1. We develop a QoS-aware adaptive scheduling algorithm that exploits the unique characteristics of LLM inference to co-schedule requests belonging to multiple QoS classes on shared infrastructure, improving throughput while maintaining latency guarantees.
2. We design and implement a hybrid prioritization and eager relegation policy that minimizes SLO violations under both optimal load and overload conditions.
3. We evaluate NIYAMA on various workloads and scenarios, demonstrating up to 32% higher serving capacity while meeting QoS guarantees compared to current approaches.

The rest of the paper is structured as follows. (§2) outlines the need for QoS-based serving systems and (§3) details the architecture and implementation of NIYAMA. (§4) presents our evaluation methodology and results.

2 Background and Motivation

2.1 LLM Inference

Large language model (LLM) inference is fundamentally different from traditional computing workloads, characterized by two distinct computational phases that significantly impact system design: the prefill and decode stages. During the prefill phase, the entire input prompt is processed simultaneously, making it computationally intensive. The subsequent decode phase generates output tokens auto-regressively, with each token’s generation depending on the previously generated tokens.

Scheduling. In this work, we assume co-located LLM inference scheduling as seen in popular serving frameworks like vLLM [14] and SGLang [27] where prefills and decodes of a request are executed on the same replica using chunked prefills [4] for better serving efficiency. Chunked prefills split a prefill request into equal-sized chunks, allowing for efficient batching and scheduling without pausing ongoing decodes. This approach helps balance the trade-off between throughput and latency, and is used as a standard scheduling practice in production systems [23].

Latency metrics. LLM inference encompasses three primary latency metrics, which serve as critical performance indicators across different application types:

1. **Time to First Token (TTFT).** This metric captures the initial response latency, measuring the duration from request submission to generating the first output token. For interactive applications like chatbots, coding assistants, or real-time conversational interfaces, TTFT is crucial as

在这种需求激增期间，吞吐量、延迟和公平性之间的复杂权衡。

在本文中，我们提出了 Niyama，一种 QoS 驱动的 LLM 推理服务系统，它通过两个关键思想解决了这些限制。首先，Niyama 支持细粒度的 QoS 类别，允许应用程序精确指定其延迟要求。通过在共享而非孤立的基础设施上共同调度具有不同 QoS 目标的请求，可以有效地服务于多个 QoS 类别。其次，Niyama 实施混合优先级和热切降级策略，允许在过载情况下优雅地降级服务。图 1 将 Niyama 与最先进的 Sarathi-Serve [4] 孤立部署进行了比较，展示了显著的性能改进。

在共享服务实例上有效支持多个 QoS 类别提出了重大挑战。一种方法是使用满足所有服务实例上最严格 QoS 类别的延迟约束所需的最小块大小。然而，这将导致所有服务类别的吞吐量较低 [4] 和成本较高。相反，Niyama 利用 LLM 推理的独特执行特性，特别是不同的预填充和解码阶段以及预填充阶段固有的可预测性，根据观察到的系统状态和单独的 QoS 目标动态调整块大小。共同服务多个 QoS 类别使我们能够利用具有宽松延迟要求的请求的截止时间裕度来调度更大块大小的突发，从而机会性地提高吞吐量。我们的评估表明，与当前的孤立部署相比，Niyama 的有效服务容量提高了 32%，同时始终满足所有请求类别的 QoS 保证。

为了优雅地管理过载情况，尼亚玛采用了混合优先级和急切的降级政策。最短作业优先 (SJF) 等简单的过载处理方法通过优先考虑短请求来管理过载。由于请求长度对 LLM 系统负载的二次依赖性，这有助于减少负载 [22]。然而，SJF 忽略了较长作业的 QoS 要求，导致即使在低负载下也会违反 SLO (图 2)。另一方面，较早截止时间优先 (EDF) 调度在低负载下是最佳的，但即使在负载略高于容量时也会遭受过多的违规。因此，Niyama 引入了一种混合策略，可以在 EDF 和 SJF 之间平滑插入，从而允许部署在低负载和高负载下最大限度地减少 SLO 违规。此外，Niyama 主动采用急切降级，有选择地降低一小部分请求的服务，以确保稳定的性能，即使在极端负载条件下也是如此。在多 QoS 场景中，Niyama 利用应用程序提供的有关请求重要性的提示（例如请求是来自免费层还是付费层）来执行降级。这可确保在过载情况下低优先级请求首先受到影响，从而使系统能够维持大多数高优先级请求的 QoS

请求。我们的评估表明，在严重过载的情况下（比容量高出 50%），Niyama 始终满足超过 95% 的请求的延迟目标，从而在依赖 LLM 基础设施的不同应用程序中节省大量成本并增强用户体验。

我们的工作做出了以下主要贡献：

1. 我们开发了一种 QoS 感知自适应调度算法，该算法利用 LLM 推理的独特特性来共同调度属于共享基础设施上多个 QoS 类别的请求，从而提高吞吐量，同时保持延迟保证。
2. 我们设计并实施混合优先级和急切降级策略，在最佳负载和过载条件下最大限度地减少 SLO 违规。
3. 我们在各种工作负载和场景下评估 Niyama，与当前方法相比，在满足 QoS 保证的同时，服务容量提高了 32%。

本文的其余部分结构如下。(§2) 概述了基于 QoS 的服务系统的需求，(§3) 详细介绍了 Niyama 的架构和实现。(§4) 介绍了我们的评估方法和结果。

2 背景和动机

2.1 大语言模型

大型语言模型 (LLM) 推理与传统计算工作负载有根本不同，其特点是两个明显影响系统设计的不同计算阶段：预填充和解码阶段。在预填充阶段，整个输入提示会同时处理，这使得计算量很大。随后的解码阶段自动回归生成输出令牌，每个令牌的生成取决于先前生成的令牌。

调度。在这项工作中，我们假设共同定位 LLM 推理调度，如 vLLM [14] 和 SGLang [27] 等流行服务框架中所示，其中请求的预填充和解码使用分块预填充 [4] 在同一副本上执行，以获得更好的服务效率。分块预填充将预填充请求分割成大小相等的块，从而实现高效的批处理和调度，而无需暂停正在进行的解码。这种方法有助于平衡吞吐量和延迟之间的权衡，并被用作生产系统中的标准调度实践 [23]。

延迟指标。LLM 推理包含三个主要延迟指标，它们作为不同应用程序类型的关键性能指标：

1. 首次令牌时间 (TTFT)。该指标捕获初始响应延迟，测量从请求提交到生成第一个输出令牌的持续时间。对于聊天机器人、编码助手或实时对话界面等交互式应用程序，TTFT 至关重要，因为

it directly influences user perception of system responsiveness.

2. **Time Between Tokens (TBT).** This metric measures the interval between the generation of consecutive output tokens of a request, and affects the overall perceived fluidity of the response which is particularly important for interactive applications where users expect a smooth, uninterrupted stream of generated content.
3. **Time to Last Token (TTLT).** This metric focuses on the total time required to complete the entire generation process. TTLT is particularly relevant for non-interactive, batch-oriented applications such as document summarization, comprehensive research analysis, or offline content generation. In these scenarios, the overall completion time matters more than the speed of initial response or token-by-token generation.

The application’s nature determines which of these metrics take priority. User-facing, interactive applications critically depend on both TTFT and TBT, as these metrics directly impact user experience and perceived system responsiveness. In contrast, non-interactive applications primarily concern themselves with TTLT, prioritizing the total time to generate a complete output over the speed of initial token generation.

2.2 Production Deployment Landscape

Due to the fundamental differences in workload characteristics and performance requirements between these application types, current industrial practices for LLM inference deployment predominantly employ a siloed infrastructure model [13], maintaining two distinct GPU clusters: (1) a dedicated fleet for latency-sensitive, interactive requests, and (2) a separate cluster for batch processing and background jobs.

Overload management. When faced with traffic exceeding capacity, current systems employ limited and often ineffective overload management techniques.

1. **Rate Limiting:** These mechanisms simply reject excess requests without considering their relative importance or potential impact.
2. **Short Request Prioritization:** These techniques favor shorter requests, which can unfairly disadvantage longer but potentially more important queries.

Such approaches are unable to provide application-aware or graceful service degradation, resulting in either uniform performance degradation across workloads or complete rejection of a class of requests without any fairness guarantees.

2.3 Deployment Challenges

Current LLM deployments create significant operational inefficiencies due to the siloed infrastructure model.

Resource provisioning and utilization. As workload demands fluctuate, dedicated clusters often operate well below

their maximum capacity, resulting in substantial resource underutilization. An interactive cluster might be overwhelmed during peak hours, while a batch processing fleet remains largely idle, leading to inefficient computational resource allocation. The complexity intensifies when supporting applications with multiple different latency requirements. Each unique performance profile potentially necessitates a dedicated infrastructure cluster, which can increase operational complexity significantly. What begins as a straightforward architectural decision quickly transforms into a management challenge, with each new cluster introducing additional capacity provisioning challenges and monitoring overhead.

Lack of graceful service degradation. Existing mechanisms for overload management, such as user rate limiting and prioritizing short requests, are often unfair and not application-aware, and thus lack an ability to gracefully degrade QoS. These techniques can lead to poor user experiences and inefficient resource utilization.

2.4 Analysis of Multi-SLA scheduling policies for LLM Inference

A practical approach to mitigating the operational complexities and resource inefficiencies of siloed infrastructure is to co-schedule requests from various applications within a unified cluster. In this section, we examine the effects of traditional scheduling policies from the literature on multi-tenant scheduling and assess their performance for LLM inference across three key dimensions: latency, SLO violations, and the fairness of SLO violations. This analysis highlights the necessity for a novel multi-tenant, SLO-aware scheduling policy tailored for LLM inference.

Scheduling policies. We compare four different scheduling policies from the literature for multi-tenant systems. First-Come-First-Served (FCFS) represents the most basic approach, processing requests in the order they arrive. More advanced policies include Shortest Job First (SJF), which prioritizes jobs with the shortest expected execution time, and Shortest Remaining Prompt First (SRPF), which continuously re-evaluates and preempts jobs to minimize overall waiting time, based on the outstanding prompt tokens to be processed. Finally, Earliest Deadline First (EDF) schedules jobs based on their impending deadlines.

Figure 2 compares the multiple scheduling policies and plots the (a) median and (b) p99 latency of requests in the system, (c) percentage of requests that violated their SLO, and (d) the number of long requests (requests with prompt length in the 90th percentile of the dataset) that violated their SLO. Despite their theoretical foundations, we observe that these scheduling approaches fundamentally struggle when applied to large language model (LLM) inference workloads. NİYAMA exploits the unique computational characteristics of LLMs — including variable input complexity, distinct prefill and decode phases, and the predictability of the prefill phase

它直接影响用户对系统响应能力的感知。

2. 令牌之间的时间 (TBT)。该指标测量请求的连续输出令牌生成之间的间隔，并影响响应的整体感知流动性，这对于用户期望生成流畅、不间断的内容流的交互式应用程序尤其重要。

3. 最后令牌时间 (TTLT)。该指标重点关注完成整个生成过程所需的总时间。TTLT 特别适用于非交互式、面向批处理的应用程序，例如文档摘要、综合研究分析或离线内容生成。在这些场景中，总体完成时间比初始响应或逐个令牌生成的速度更重要。

应用程序的性质决定了这些指标中的哪一个具有优先权。面向用户的交互式应用程序严重依赖 TTFT 和 TBT，因为这些指标直接影响用户体验和感知的系统响应能力。相比之下，非交互式应用程序主要关注 TTLT，优先考虑生成完整输出的总时间而不是初始令牌生成的速度。

2.2 生产部署格局

由于这些应用程序类型之间的工作负载特征和性能要求存在根本差异，当前 LLM 推理部署的工业实践主要采用孤立的基础设施模型 [13]，维护两个不同的 GPU 集群：（1）用于延迟敏感、交互式请求的专用集群，以及（2）用于批处理和后台作业的单独集群。

超载管理。当面临超出容量的流量时，当前系统采用有限且通常无效的过载管理技术。

1. 速率限制：这些机制只是简单地拒绝多余的请求，而不考虑它们的相对重要性或潜在影响。
2. 短请求优先级：这些技术有利于较短的请求，这可能不公平地使较长但可能更重要的查询处于不利地位。

此类方法无法提供应用程序感知或优雅的服务降级，从而导致跨工作负载的统一性能降级或在没有任何公平保证的情况下完全拒绝一类请求。

2.3 部署挑战

由于孤立的基础设施模型，当前的大语言模型导致运营效率显著低下。

资源配置和利用。随着工作负载需求的波动，专用集群的运行速度通常远低于

其最大容量，导致大量资源未得到充分利用。交互式集群在高峰时段可能会不堪重负，而批处理队列基本上处于闲置状态，从而导致计算资源分配效率低下。当支持具有多种不同延迟要求的应用程序时，复杂性就会加剧。每个独特的性能配置文件都可能需要专用的基础设施集群，这会显著增加操作复杂性。最初的简单架构决策很快就转变为管理挑战，每个新集群都会引入额外的容量配置挑战和监控开销。

缺乏优雅的服务降级。现有的过载管理机制（例如用户速率限制和短请求优先级）通常是不公平的并且不具有应用程序感知能力，因此缺乏优雅地降低 QoS 的能力。这些技术可能会导致糟糕的用户体验和低效的资源利用。

2.4 LLM Inference的多SLA调度策略分析

减轻孤立基础设施的操作复杂性和资源效率低下的实用方法是在统一集群内共同调度来自各种应用程序的请求。在本节中，我们从有关多租户调度的文献中研究传统调度策略的影响，并评估其在三个关键维度上的 LLM 推理性能：延迟、SLO 违规和 SLO 违规的公平性。此分析强调了为 LLM 推理量身定制的新型多租户、SLO 感知调度策略的必要性。

调度策略。我们比较了多租户系统文献中的四种不同的调度策略。先来先服务 (FCFS) 代表最基本的方法，按照请求到达的顺序处理请求。更高级的策略包括最短作业优先 (SJF) 和最短剩余提示优先 (SRPF)，前者优先考虑预期执行时间最短的作业，后者根据要处理的未完成的提示令牌，不断重新评估和抢占作业，以最大限度地减少总体等待时间。最后，最早截止日期优先 (EDF) 根据即将到来的截止日期来安排作业。

图 2 比较了多种调度策略，并绘制了系统中请求的 (a) 中值和 (b) p99 延迟、(c) 违反其 SLO 的请求的百分比，以及 (d) 违反其 SLO 的长请求（提示长度位于数据集第 90 个百分位数的请求）的数量。尽管有理论基础，但我们观察到，这些调度方法在应用于大型语言模型 (LLM) 推理工作负载时从根本上来说很困难。Niyama 利用了大语言模型的计算特性，包括可变输入复杂性、不同的预填充和解码阶段以及预填充阶段的可预测性

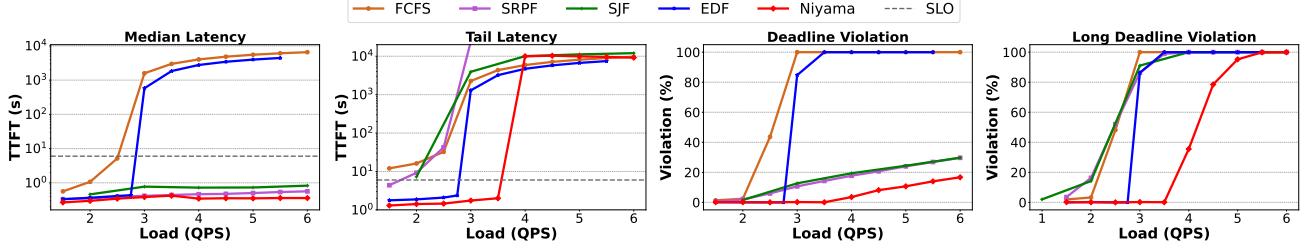


Figure 2. Comparison of traditional policies for multi-SLA scheduling. The graphs plot the latency and violations in the strictest QoS class. FCFS breaks down very quickly because urgent requests can be blocked by non-urgent ones. Deadline-aware policies like EDF are better than FCFS, but cannot gracefully degrade at high loads because of intense queue buildup. SJF/SRPF on the other hand can maintain QoS in the median case but violates SLOs of majority of long jobs even at a low load of 2.5 QPS. NIYAMA interpolates smoothly between SJF and EDF and minimizes violations across all load conditions.

to devise an SLO-aware scheduling policy which minimizes latency and SLO violations while maximizing throughput, as we show in our evaluations (Figures 8 and 9).

Summary. In this paper, we address these critical infrastructure challenges by introducing a QoS-aware serving framework, NIYAMA. Our system transforms LLM inference serving from a static, siloed approach to a dynamic, application-aware computational system. By introducing sophisticated service level objective (SLO) management, NIYAMA enables more efficient, responsive, and cost-effective infrastructure for next-generation AI applications.

3 NIYAMA: Design and Implementation

NIYAMA is designed to efficiently manage concurrent LLM inference requests with diverse QoS requirements, while maximizing resource utilization across the shared infrastructure. We address the limitations outlined earlier by dynamically adapting scheduling decisions based on real-time system state and QoS targets of the in-flight requests.

3.1 Overview

The architecture of NIYAMA is shown in Figure 3. A request in NIYAMA can be in one of three queues — 1) prefill queue, 2) decode queue, or 3) relegated queue. ① When a request enters the system, it is put into the prefill queue. In each iteration, NIYAMA constructs a batch consisting of all requests in the decode queue, and a prefill-chunk from a request in the prefill queue. The prefill selector uses *hybrid prioritization* to select the prefill request for the current batch. ② The violation checker module validates that the chosen request has not already (or will) violate its QoS targets in the current iteration. ③ If it does, it is eagerly moved into the relegated queue and a different prefill request is chosen. The relegated requests are serviced opportunistically during periods of lower system load, ensuring eventual completion without permanent rejection; while enabling graceful degradation under overload conditions. ④ A lightweight predictor is then used to estimate the latency of the batch to make sure that

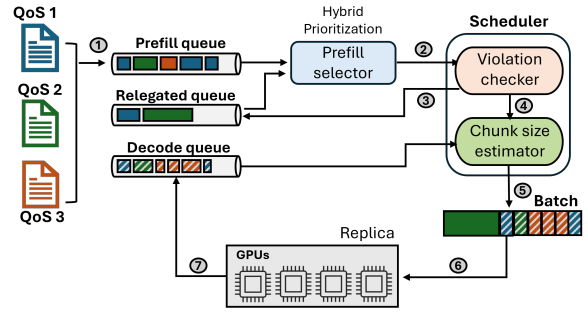


Figure 3. Overview of NIYAMA

the QoS targets are not violated, while maximizing the chunk size for efficiency. ⑤ A mixed batch of prefill and decode tokens is constructed using the chosen prefill chunk and the requests in the decode queue, which is then ⑥ dispatched to the execution engine on the GPU for processing. ⑦ Once the prefill portion of a request is completed, it is moved to the decode queue, and subsequent iterations continue.

3.2 QoS Classes and Deadlines

NIYAMA defines two QoS classes: interactive and non-interactive. Interactive requests use two SLOs — TTFT (time to first token) and TBT (token-by-token latency), which ensure immediate responsiveness and consistent pacing. Non-interactive requests have a single TTLT (total latency) target, focused on overall completion. Although, we define two QoS classes, the application owner is free to specify their custom SLO targets within the class, allowing for flexibility and customization to specific application needs as shown in Table 2.

The deadline for each request is determined based on its QoS class. For the interactive QoS class, following the approach in [2], the deadline for the first token is defined as:

$$D_{first} = t_{arrival} + SLO_{TTFT}, \quad (1)$$

while subsequent tokens' deadlines are calculated using:

$$D_n = t_{arrival} + SLO_{TTFT} + (n - 1) \cdot SLO_{TBT}, \quad (2)$$

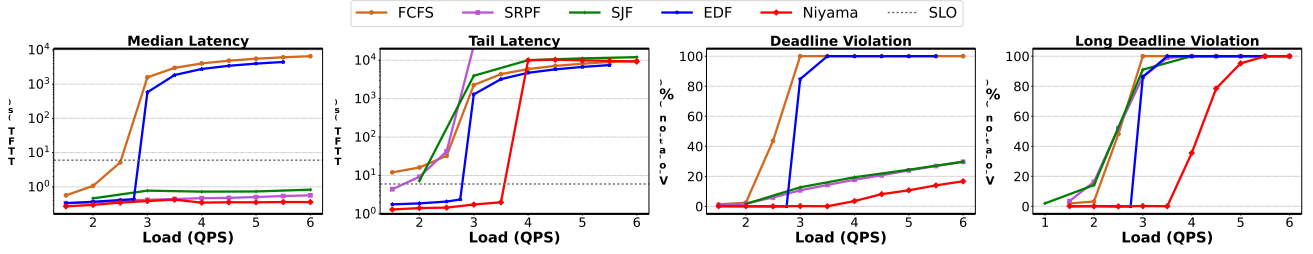


图 2. 多 SLA 调度传统策略的比较。这些图表绘制了最严格的 QoS 类别中的延迟和违规情况。FCFS 崩溃得很快，因为紧急请求可能会被非紧急请求阻止。EDF 等截止日期感知策略比 FCFS 更好，但由于队列堆积密集，无法在高负载下正常降级。另一方面，SJF/SRPF 可以在中等情况下维持 QoS，但即使在 2.5 QPS 的低负载下，也会违反大多数数长作业的 SLO。Niyama 在 SJF 和 EDF 之间平滑插值，并最大限度地减少所有负载条件下的违规。

设计一个 SLO 感知调度策略，最大限度地减少延迟和 SLO 违规，同时最大化吞吐量，正如我们在评估中所示（图 8 和 9）。

概括。在本文中，我们通过引入 QoS 感知服务框架 Niyama 来解决这些关键的基础设施挑战。我们的系统将 LLM 推理服务从静态、孤立的方法转变为动态、应用程序感知的计算系统。通过引入复杂的服务水平目标 (SLO) 管理，Niyama 为下一代 AI 应用程序提供了更高效、响应更快且更具成本效益的基础设施。

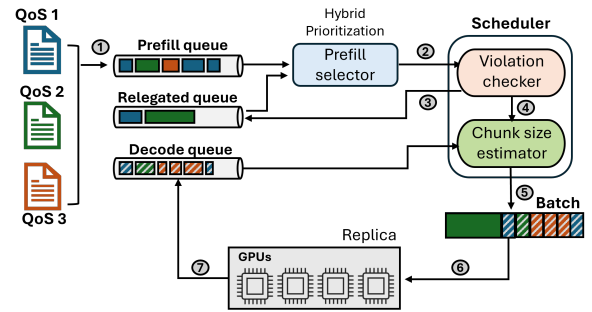


图 3. 尼亚玛 (Niyama) 概览

3 Niyama：设计与实现

Niyama 旨在有效管理具有不同 QoS 要求的并发 LLM 推理请求，同时最大限度地提高共享基础设施的资源利用率。我们通过根据实时系统状态和正在进行的请求的 QoS 目标动态调整调度决策来解决前面概述的限制。

3.1 概述

Niyama 的架构如图 3 所示。Niyama 中的请求可以位于三个队列之一：1) 预填充队列、2) 解码队列或 3) 降级队列。1 当请求进入系统时，它被放入预填充队列中。在每次迭代中，Niyama 都会构建一个由解码队列中的所有请求组成的批次，以及来自预填充队列中的请求的预填充块。预填充选择器使用混合优先级来选择当前批次的预填充请求。2 违规检查器模块验证所选请求在当前迭代中尚未（或将）违反其 QoS 目标。3 如果是，则立即将其移动到降级队列中，并选择不同的预填充请求。在系统负载较低期间，会适时地处理降级的请求，确保最终完成而不会永久拒绝；同时在过载条件下实现优雅降级。4 然后使用轻量级预测器来估计批次的延迟，以确保

不违反 QoS 目标，同时最大化块大小以提高效率。5 使用选定的预填充块和解码队列中的请求构建预填充和解码令牌的混合批次，然后将其分派到 GPU 上的执行引擎进行处理 6。7 一旦请求的预填充部分完成，它就会移至解码队列，并继续后续迭代。

3.2 QoS 等级和期限

Niyama 定义了两个 QoS 类别：交互式和非交互式。交互式请求使用两个 SLO——TTFT（第一个令牌的时间）和 TBT（逐个令牌的延迟），确保立即响应和一致的节奏。非交互式请求有一个 TTLT（总延迟）目标，重点关注整体完成情况。尽管我们定义了两个 QoS 类，但应用程序所有者可以在类中自由指定其自定义 SLO 目标，从而实现针对特定应用程序需求的灵活性和自定义，如表 2 所示。

每个请求的截止时间根据其 QoS 类别确定。对于交互式 QoS 类，按照[2]中的方法，第一个令牌的截止时间定义为：

$$D_{first} = t_{arrival} + SLO_{TTFT}, \quad (1)$$

而后续代币的截止日期是使用以下方式计算的：

$$D_n = t_{arrival} + SLO_{TTFT} + (n - 1) \cdot SLO_{TBT}, \quad (2)$$

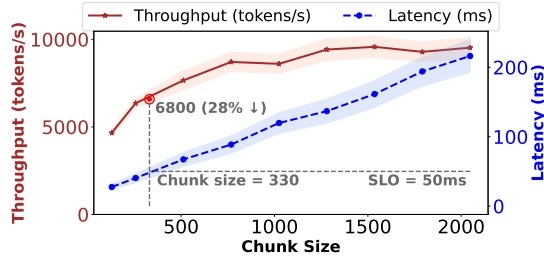


Figure 4. Performance characteristics as a function of chunk size, showing the throughput-latency tradeoff.

where n is the token position. For non-interactive requests a deadline is set only for the full completion of the request as:

$$D_{total} = t_{arrival} + SLO_{TTLT} \quad (3)$$

Once we have defined the deadlines for each request, NIYAMA scheduling aims to minimize deadline violations while maximizing throughput.

3.3 Dynamic Chunking

State-of-the-art LLM inference serving frameworks [4, 14] serve requests using chunked-prefills, where each iteration processes a fixed number of tokens (called chunk size), which includes both prefill and decode tokens from different requests using fused prefill-decode MLP to improve the compute efficiency of memory-bound decode phase [5]. However, this involves a fundamental trade-off between throughput and latency – a larger chunk results in better throughput but increases the TBT of the decodes in the batch. This is illustrated in Figure 4.

A naïve approach for co-scheduling jobs of different QoS classes with varying deadlines on TTFT, TBT, and TTLT would be to use the smallest chunk size necessary to meet the latency constraint of the strictest QoS class. However, this results in low throughput and high cost for all service classes.

NIYAMA employs *dynamic chunking* to opportunistically maximize the chunk size for the prefill request by exploiting any available slack in the deadlines of the requests being currently serviced. For each request in the decode queue, we define slack as the difference between the deadline for the next token (eq 2) and current time. Using this slack and characteristics of the requests in decode phase, we calculate the maximum chunk size which can be used for the current batch without violating any deadline.

3.4 NIYAMA Scheduling

While dynamic chunking allows us to choose an optimal chunk size for a prefill request, we also need to decide which request from the prefill queue should be processed in the current scheduling iteration.

Hybrid Prioritization. As shown in Figure 2, existing scheduling policies struggle with LLM workloads at higher loads. For example, EDF which prioritizes requests with earlier deadline has very low deadline violation rates (Figure 2(c)) at low loads, but the violation rates spike to almost 100% when the load goes beyond a threshold. On the other hand, policies which prioritize short work requests – SRPF and SJF – handle higher loads much better but are worse than EDF at lower loads. Further, SRPF and SJF achieve this at the expense of unfairly penalizing long jobs (Figure 2(d)) without any regard to the request priorities while EDF does not suffer from this problem. To handle varying load conditions which is common in production services and maintain fairness across requests, our first key insight is a *hybrid prioritization* scheme which interpolates between SRPF and EDF. This allows us to get EDF characteristics at low loads, and leverage SRPF semantics under overload conditions while maintaining fairness.

To implement this scheduling, NIYAMA smoothly interpolates between EDF and SRPF to compute the priority of a request. For interactive requests, the priority is computed by taking a linear combination of the TTFT deadline (this incorporates EDF semantics) and the estimated time taken which will be needed to process the remaining prefills (this incorporates SRPF semantics) of the request as:

$$P^i = t_{arrival}^i + SLO_{TTFT}^i + \alpha * Prefill_{rem}^i. \quad (4)$$

Note that we only consider TTFT deadline as meeting TBT deadlines is taken care by our dynamic chunking scheme. For non-interactive requests, the priority is computed as

$$P^i = t_{arrival}^i + SLO_{TTLT}^i + \alpha * (Prefill_{rem}^i + Decode_{rem}^i), \quad (5)$$

where $Decode_{rem}$ indicates the time to compute all the decode tokens. Since decode length is unknown in LLM inference, this introduces a challenge in modeling the priority of non-interactive requests. We address this with a simple insight – for non-interactive jobs, the TTLT deadline is typically much greater than the actual processing time. Therefore, given an application, we can use historic information on the decode tokens generated by that application and overapproximate it by two standard deviations. We show in (§6) that this simple prediction sufficiently captures the priority of non-interactive jobs.

Eager Relegation. Our *hybrid prioritization* scheduling strikes a balance between minimizing deadline violations and fairness. However, as shown in Figure 5, under overload conditions, NIYAMA (or any scheduling policy) still cannot service all incoming requests at the desired QoS SLOs. Our second key insight is that by *eagerly relegating* a small fraction of requests that we know will miss their deadlines, one can provide stable performance for the majority, enabling graceful service degradation under overload conditions. The key idea is simple – if a request has already violated its TTFT /

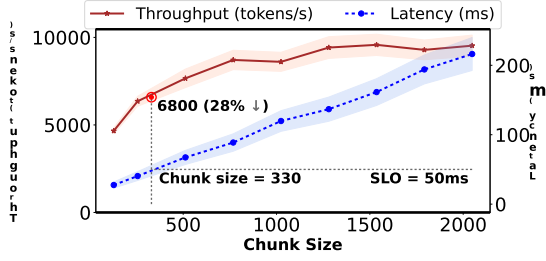


图 4. 性能特征作为块大小的函数，显示了吞吐量与延迟的权衡。

其中 n 是令牌位置。对于非交互式请求，仅为完全完成请求设置截止日期，如下所示：

$$D_{total} = t_{arrival} + SLO_{TTLT} \quad (3)$$

一旦我们定义了每个请求的截止日期，Niyama 调度的目标就是最大限度地减少截止日期违规，同时最大化吞吐量。

3.3 动态分块

最先进的 LLM 推理服务框架 [4, 14] 使用分块预填充来处理服务请求，其中每次迭代处理固定数量的令牌（称为块大小），其中包括使用融合预填充-解码 MLP 的不同请求的预填充和解码令牌，以提高内存绑定解码阶段的计算效率 [5]。然而，这涉及吞吐量和延迟之间的基本权衡——较大的块会带来更好的吞吐量，但会增加批次中解码的 TBT。图 4 对此进行了说明。

在 TTFT、TBT 和 TTLT 上共同调度具有不同截止时间的不同 QoS 类别的作业的一种简单方法是使用满足最严格 QoS 类别的延迟约束所需的最小块大小。然而，这会导致所有服务类别的吞吐量低且成本高。

Niyama 采用动态分块，通过利用当前正在服务的请求的最后期限中的任何可用间隙，机会最大化预填充请求的块大小。对于解码队列中的每个请求，我们将松弛定义为下一个令牌的截止时间 (eq 2) 与当前时间之间的差异。利用这个松弛和解码阶段请求的特征，我们计算可用于当前批次的最大块大小，而不会违反任何截止日期。

3.4 尼亚玛调度

虽然动态分块允许我们为预填充请求选择最佳块大小，但我们还需要决定应在当前调度迭代中处理预填充队列中的哪个请求。

混合优先级。如图 2 所示，现有的调度策略难以应对较高负载下的 LLM 工作负载。例如，优先考虑较早截止日期的请求的 EDF 在低负载时具有非常低的截止日期违规率（图 2(c)），但当负载超过阈值时，违规率几乎达到 100%。另一方面，优先考虑短工作请求的策略（SRPF 和 SJF）可以更好地处理较高负载，但在较低负载下比 EDF 更差。此外，SRPF 和 SJF 实现这一目标的代价是不公平地惩罚长作业（图 2(d)），而不考虑请求优先级，而 EDF 则不会遇到此问题。为了处理生产服务中常见的不同负载条件并保持请求之间的公平性，我们的第一个关键见解是在 SRPF 和 EDF 之间插值的混合优先级方案。这使我们能够在低负载下获得 EDF 特性，并在过载条件下利用 SRPF 语义，同时保持公平性。

为了实现这种调度，Niyama 在 EDF 和 SRPF 之间平滑地进行插值以计算请求的优先级。对于交互式请求，优先级是通过 TTFT 截止时间（这包含 EDF 语义）和处理请求的剩余预填充（这包含 SRPF 语义）所需的估计时间的线性组合来计算的，如下所示：

$$P^i = t_{arrival}^i + SLO_{TTFT}^i + \alpha * Prefill_{rem}^i \quad (4)$$

请注意，我们只考虑 TTFT 截止日期，因为我们的动态分块方案会满足 TBT 截止日期。对于非交互式请求，优先级计算如下

$$P^i = t_{arrival}^i + SLO_{TTLT}^i + \alpha * (Prefill_{rem}^i + Decode_{rem}^i), \quad (5)$$

其中 $Decode_{rem}$ 表示计算所有解码令牌的时间。由于 LLM 推理中解码长度是未知的，这给非交互式请求的优先级建模带来了挑战。我们用一个简单的见解来解决这个问题——对于非交互式作业，TTLT 截止日期通常比实际处理时间长得多。因此，对于给定的应用程序，我们可以使用该应用程序生成的解码令牌的历史信息，并通过两个标准差对其进行过度近似。我们在 (§6) 中表明，这个简单的预测足以捕捉非交互式工作的优先级。

急于保级。我们的混合优先级调度在最大限度地减少最后期限违规和公平性之间取得了平衡。然而，如图 5 所示，在过载条件下，Niyama（或任何调度策略）仍然无法以所需的 QoS SLO 为所有传入请求提供服务。我们的第二个关键见解是，通过急切地降级我们知道会错过最后期限的一小部分请求，可以为大多数请求提供稳定的性能，从而在过载条件下实现优雅的服务降级。关键思想很简单——如果一个请求已经违反了它的 TTFT /

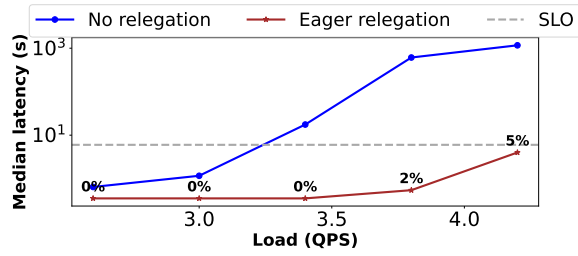


Figure 5. Proactively relegating a small percentage of requests enormously helps in maintaining the quality of service for the median request in the system, which otherwise grows exponentially due to a cascade of violations.

TTLT deadline, or is about to violate it in the current iteration, then NIYAMA de-prioritizes this request into a relegated queue. In multi-tenant deployments, we also use application hints such as free vs paid tier to preferentially relegate low-priority requests to ensure stability of service to the high priority ones. Only when there are no more low-priority requests, NIYAMA proactively relegates high-priority requests that have violated their deadlines to prevent cascading deadline violations. This enables graceful degradation of service even under extreme load. As shown in Figure 5, by relegating just 5% of the requests, we can maintain latency SLOs even under very high overload conditions.

Selective Preemption. Note that our *hybrid prioritization* scheduling can preempt an in-flight request for which a few prefill chunks have already been processed to instead service a new incoming request with a strict QoS target with a higher computed priority (eq 4). Preemption is a desirable capability as it avoids head-of-line blocking of small interactive requests behind long batch requests. However, in LLM serving, the memory overhead of preemption can be significant as the KV-cache of requests can be quite large. To avoid this, NIYAMA uses *selective preemption*, where we preempt a request to accommodate another with a higher priority only if (1) the in-flight request is in the prefill queue (i.e., requests in the decode queue are never preempted), and (2) preempting that request for an iteration does not result in its deadline violation. We do not preempt requests in the decode queue as TBT latency targets are typically very strict (10s of milliseconds), and thus preempting them significantly increases the chances of TBT violation. This approach also ensures that the KV-cache for each request remains in the GPU for the shortest necessary duration, thereby minimizing memory pressure.

3.5 An Illustrative Example

Figure 6 illustrates our approach with an example of five requests (A–E) across three QoS buckets. In this example, A is

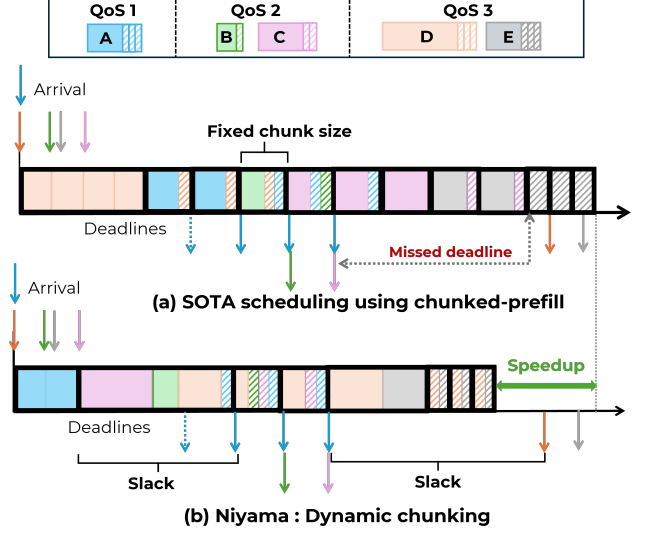


Figure 6. An illustration of how NIYAMA improves throughput using dynamic chunking compared to SOTA scheduling.

an interactive request while others are non-interactive. State-of-the-art LLM schedulers like vllm [14] and sarathi [4] will execute each iteration using a fixed chunk size and process requests in arrival order (FCFS). Our solution introduces two key improvements.

First, we prioritize requests based on their QoS targets using our hybrid prioritization, which will prioritize request A before D due to its earlier deadline. Second, we dynamically adjust chunk sizes based on accumulated slack. For example, after A’s prefill phase completes earlier than its deadline, it accumulates significant slack before its next token is due. We exploit this slack by dynamically increasing the chunk size, adding more prefill tokens from requests B and D (which have the earliest deadlines in the queue), thereby improving throughput without violating any ongoing request deadlines.

When the interactive job A enters its decode phase, we revert to the original smaller chunk size necessary to meet its TBT requirement, though we still exploit any slack accumulated if decoding completes faster than predicted. Once A completes and no remaining requests impose strict TBT constraints, we again increase chunk size to maximize throughput while respecting the TTLT deadlines of ongoing requests. This approach effectively leverages the predictable execution characteristics of LLMs to dynamically optimize chunk sizes during runtime, balancing throughput and deadline requirements.

3.6 Implementation

We implemented NIYAMA by extending the Sarathi scheduler [4], which is built on top of the vLLM inference system [14]. The implementation focuses on enhancing the scheduler component while maintaining compatibility with

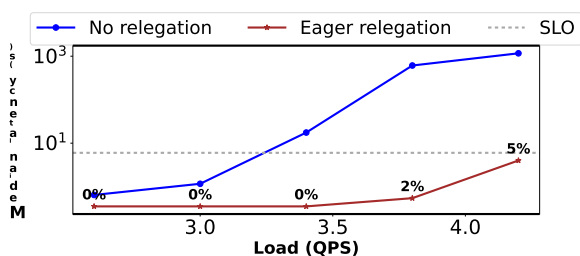


图 5. 主动降级一小部分请求极大地有助于维持系统中中等请求的服务质量，否则该请求会因级联违规而呈指数级增长。

TTLT 截止时间，或者在当前迭代中即将违反它，然后 Niyama 会取消该请求的优先级，将其放入降级队列中。在多租户部署中，我们还使用免费与付费级别等应用程序提示来优先降级低优先级请求，以确保高优先级请求的服务稳定性。仅当不再有低优先级请求时，Niyama 才会主动降级违反截止日期的高优先级请求，以防止级联违反截止日期的情况。即使在极端负载下，这也能实现服务的平稳降级。如图 5 所示，通过仅降级 5% 的请求，即使在非常高的过载条件下，我们也可以维持延迟 SLO。

选择性抢占。请注意，我们的混合优先级调度可以抢占已处理了一些预填充块的正在进行的请求，以使用具有更高计算优先级的严格 QoS 目标来服务新的传入请求（等式 4）。抢占是一项理想的功能，因为它可以避免在长批量请求后面的小型交互式请求的队头阻塞。然而，在 LLM 服务中，抢占的内存开销可能很大，因为请求的 KV 缓存可能非常大。为了避免这种情况，Niyama 使用选择性抢占，只有在 (1) 进行中的请求位于预填充队列中（即解码队列中的请求永远不会被抢占），并且 (2) 抢占该迭代请求不会导致其违反截止时间时，我们才会抢占一个请求以容纳另一个具有更高优先级的请求。我们不会抢占解码队列中的请求，因为 TBT 延迟目标通常非常严格（数十毫秒），因此抢占它们会显着增加 TBT 违规的机会。这种方法还确保每个请求的 KV 缓存在 GPU 中保留最短的必要持续时间，从而最大限度地减少内存压力。

3.5 一个说明性例子

图 6 通过跨三个 QoS 桶的五个请求 (A-E) 示例说明了我们的方法。在这个例子中，A 是

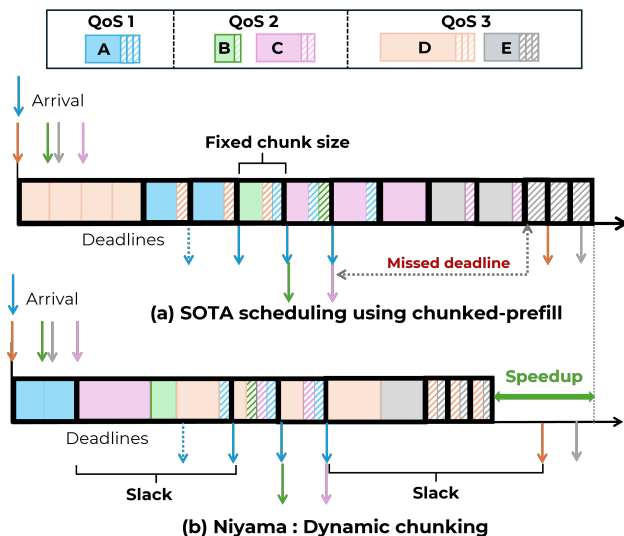


图 6. 与 SOTA 调度相比，Niyama 如何使用动态分块提高吞吐量的图示。

交互式请求，而其他请求则为非交互式。最先进的 LLM 调度程序（如 vllm [14] 和 sarathi [4]）将使用固定的块大小执行每次迭代，并按到达顺序（FCFS）处理请求。我们的解决方案引入了两项关键改进。

首先，我们使用混合优先级根据请求的 QoS 目标对请求进行优先级排序，由于截止日期较早，因此会将请求 A 优先于 D。其次，我们根据累积的松弛动态调整块大小。例如，在 A 的预填充阶段早于截止日期完成后，它会在下一个令牌到期之前积累大量的闲置时间。我们通过动态增加块大小、从请求 B 和 D（队列中具有最早的截止日期）添加更多预填充令牌来利用这一空闲时间，从而在不违反任何正在进行的请求截止日期的情况下提高吞吐量。

当交互式作业 A 进入解码阶段时，我们恢复到满足其 TBT 要求所需的原始较小块大小，尽管如果解码完成速度快于预期，我们仍然会利用累积的任何松弛。一旦 A 完成并且没有剩余请求施加严格的 TBT 约束，我们将再次增加块大小以最大化吞吐量，同时尊重正在进行的请求的 TTLT 截止时间。这种方法有效地利用了 LLM 的可预测执行特性，在运行时动态优化块大小，平衡吞吐量和截止时间要求。

3.6 实施

我们通过扩展 Sarathi 调度器 [4] 来实现 Niyama，该调度器构建在 vLLM 推理系统 [14] 之上。该实现的重点是增强调度程序组件，同时保持与

Dataset	Prompt tokens		Decode tokens	
	p50	p90	p50	p90
ShareGPT	1730	5696	415	834
Azure Conv	928	3830	41	342
Azure Code	1930	6251	8	43

Table 1. Datasets used in evaluation

vLLM’s efficient tensor parallelism and PagedAttention mechanisms. We extended the vLLM API to associate each inference request with its corresponding QoS requirements (TTFT, TBT, and/or TTLT) and priority level during request submission. The dynamic chunk size predictor analyzes the characteristics of requests during the decode phase and determines an optimal chunk size that maximizes throughput while adhering to latency constraints. It is implemented as a lightweight random forest model, trained on performance profiles collected from the Vidur LLM inference simulator [3], ensuring minimal overhead in the scheduling process. The hybrid prioritization policy is implemented using a priority queue that incorporates both deadline proximity and estimated processing time, with the interpolation factor α configurable as a deployment parameter. For non-interactive requests, we maintain a running history of token generation patterns per application to estimate the expected decode length. To support multi-tenant deployments, we added a priority field to each request that enables informed relegation decisions based on application hints such as free-tier versus premium users.

4 Evaluation

Our evaluation aims to answer the following questions.

1. What is the improvement due to NIYAMA in the serving capacity while meeting specified QoS SLOs (§4.1).
2. What is the impact of NIYAMA on request latencies and deadline violations under high load conditions (§4.2).
3. How does NIYAMA respond to transient spikes in load (§4.3).
4. What is the independent impact of the different optimizations and design choices used in NIYAMA (§4.4)?

Models and Hardware. We evaluate our system across two different models and parallelism strategies; Llama3-8B model deployed on a single A100 GPU with 80GB physical memory, and Qwen-7B deployed across two A100 GPUs (TP2). We evaluate them across three different popular open-source datasets which have different ratios of prefill to decode tokens, as shown in Table 1.

Workloads and QoS Tiers. For workloads, we use popular open-source datasets such as ShareGPT [24] and coding and conversation production traces from multiple LLM inference services in Azure [20]. Request arrival times are generated using a Poisson distribution, [4, 17], while maintaining the prefill and decode token counts of the respective traces. To emulate different applications, we divide each dataset into

QoS bucket	Request ratio	Interactive		Non-interactive TTLT(s)
		TTFT(s)	TBT(ms)	
Q1	33.33%	6	50	-
Q2	33.33%	-	-	600
Q3	33.33%	-	-	1800

Table 2. QoS classes and workload composition

three equal parts, and assign each part with a different application type and the corresponding QoS bucket and SLO. We consider three QoS buckets: one interactive and two non-interactive, as shown in Table 2.

Baselines. We built NIYAMA on top of Sarathi-Serve [4], which itself extends vLLM [14]. Our evaluation includes several baseline configurations: (1) **Sarathi-Silo (SOTA)**, the state-of-the-art siloed deployment where each QoS bucket is assigned an independent GPU cluster with each replica running a Sarathi scheduler (2) **Sarathi-FCFS**, which co-schedules requests across all QoS Tiers on a unified cluster using Sarathi with FCFS prioritization policy, and (3) **Sarathi-EDF**, which again co-schedules but also imparts deadline-awareness during scheduling by using the Earliest Deadline First policy on Sarathi. The strictest QoS bucket with 50ms TBT deadline uses a chunk size of 256, while the other two QoS classes use a large chunk size of 2K to maximize throughput in the siloed baselines. For shared cluster baselines, the chunk size chosen is 256, to meet the TBT targets of the strictest tier.

Setup. We first evaluate NIYAMA under uniform load conditions to identify the impact of our design on goodput (§4.1) as well as on latency and SLO violations (§4.2). Next, we evaluate how NIYAMA performs under transient spikes in load (§4.3), and finally we perform detailed ablation of our individual techniques (§4.4).

4.1 Capacity Evaluation at Uniform Load

4.1.1 Cost Efficiency from Co-located Scheduling. We measure the cost benefits due to our co-located scheme compared to the siloed baseline. For this, we compute the number of A100 GPUs required to serve a total load of 50 queries per second (QPS) spread equally across requests from the three QoS classes described in Table 2 on three different datasets. We define the serving throughput per replica as the maximum load that can be sustained while violating QoS targets for at most 1% of the total requests.

Figure 7a shows this computation for the different systems. Across different workloads, NIYAMA reduces GPU requirements by 13–32% compared to the state-of-the-art (SOTA) siloed baseline. This reduction in resources directly translates to cost savings. NIYAMA achieves better resource efficiency by maximizing throughput while meeting QoS targets through dynamic chunking. Since the lengths of requests (prompt length as well as number of decode tokens) can vary over time, even at uniform QPS the compute load on the serving system varies. NIYAMA benefits from dynamically

Dataset	Prompt tokens		Decode tokens	
	p50	p90	p50	p90
ShareGPT	1730	5696	415	834
Azure Conv	928	3830	41	342
Azure Code	1930	6251	8	43

表 1. 评估中使用的数据集

vLLM 高效的张量并行性和 PagedAttention 机制。我们扩展了 vLLM API，将每个推理请求与其相应的 QoS 要求（TTFT、TBT 和/或 TTLT）以及请求提交期间的优先级相关联。动态块大小预测器在解码阶段分析请求的特征，并确定在遵守延迟约束的同时最大化吞吐量的最佳块大小。它作为轻量级随机森林模型实现，根据从 Vidur LLM 推理模拟器 [3] 收集的性能配置文件进行训练，确保调度过程中的开销最小。混合优先级策略是使用优先级队列来实现的，该优先级队列结合了截止日期邻近度和估计处理时间，并且插值因子 α 可配置为部署参数。对于非交互式请求，我们维护每个应用程序令牌生成模式的运行历史记录，以估计预期的解码长度。为了支持多租户部署，我们为每个请求添加了一个优先级字段，以便根据应用程序提示（例如免费用户与高级用户）做出明智的降级决策。

4 评价

我们的评估旨在回答以下问题。

1. Niyama 在满足指定 QoS SLO 的同时，在服务能力方面有何改进（第 4.1 节）。
2. Niyama 对高负载条件下的请求延迟和截止日期违规有何影响（第 4.2 节）。
3. Niyama 如何响应负载的瞬态峰值（第 4.3 节）。
4. Niyama（§4.4）中使用的不同优化和设计选择的独立影响是什么？

模型和硬件。我们通过两种不同的模型和并行策略评估我们的系统；Llama3-8B 模型部署在具有 80GB 物理内存的单个 A100 GPU 上，Qwen-7B 部署在两个 A100 GPU (TP2) 上。我们在三个不同的流行开源数据集上对它们进行了评估，这些数据集具有不同的预填充与解码令牌的比率，如表 1 所示。

工作负载和 QoS 层。对于工作负载，我们使用流行的开源数据集，例如 ShareGPT [24] 以及来自 Azure [20] 中多个 LLM 推理服务的编码和对话生产跟踪。请求到达时间是使用泊松分布生成的，[4, 17]，同时维护各个跟踪的预填充和解码令牌计数。为了模拟不同的应用程序，我们将每个数据集分为

QoS bucket	Request ratio	Interactive		Non-interactive TTLT(s)
		TTFT(s)	TBT(ms)	
Q1	33.33%	6	50	-
Q2	33.33%	-	-	600
Q3	33.33%	-	-	1800

表 2. QoS 类别和工作负载组成

三个相等的部分，并为每个部分分配不同的应用程序类型以及相应的 QoS 桶和 SLO。我们考虑三种 QoS 桶：一种是交互式的，两种是非交互式的，如表 2 所示。

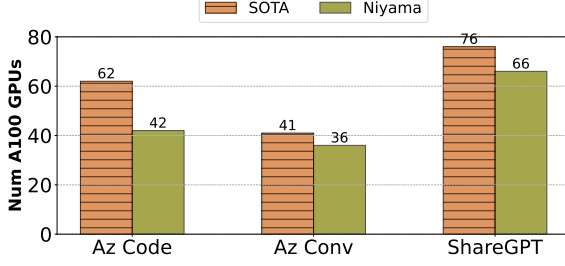
基线。我们在 Sarathi-Serve [4] 之上构建了 Niyama，它本身扩展了 vLLM [14]。我们的评估包括几个基线配置：（1）Sarathi-Silo（SOTA），最先进的孤立部署，其中每个 QoS 存储桶都分配有一个独立的 GPU 集群，每个副本运行 Sarathi 调度程序（2）Sarathi-FCFS，它使用 Sarathi 和 FCFS 优先级策略在统一集群上的所有 QoS 层上共同调度请求，以及（3）Sarathi-EDF，它再次共同安排，还通过在 Sarathi 上使用最早截止日期优先政策，在安排过程中传递截止日期意识。具有 50 毫秒 TBT 截止时间的最严格的 QoS 桶使用 256 的块大小，而其他两个 QoS 类别使用 2K 的大块大小来最大化孤立基线中的吞吐量。对于共享集群基线，选择的块大小为 256，以满足最严格层的 TBT 目标。

设置。我们首先在统一负载条件下评估 Niyama，以确定我们的设计对吞吐量（第 4.1 节）以及延迟和 SLO 违规（第 4.2 节）的影响。接下来，我们评估 Niyama 在负载瞬时峰值下的表现（第 4.3 节），最后我们对个别技术进行详细的消融（第 4.4 节）。

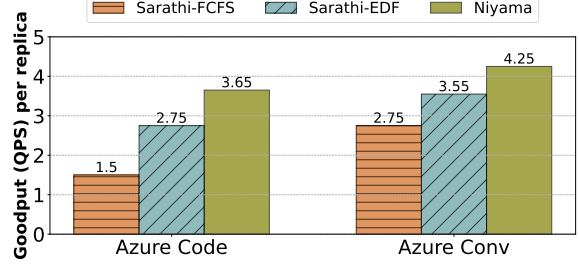
4.1 均匀负载下的容量评估

4.1.1 协同定位调度的成本效率。我们衡量了我们的同地办公方案与孤立基线相比的成本效益。为此，我们计算了为每秒 50 个查询 (QPS) 的总负载提供服务所需的 A100 GPU 数量，这些负载均匀分布在表 2 中描述的三个不同数据集上的三个 QoS 类别的请求中。我们将每个副本的服务吞吐量定义为在最多 1% 的总请求违反 QoS 目标的情况下可以维持的最大负载。

图 7a 显示了不同系统的计算结果。在不同的工作负载中，与最先进的 (SOTA) 孤立基线相比，Niyama 将 GPU 需求降低了 13-32%。资源的减少直接转化为成本的节省。Niyama 通过最大化吞吐量实现了更好的资源效率，同时通过动态分块满足 QoS 目标。由于请求的长度（提示长度以及解码令牌的数量）可能随时间变化，因此即使在统一的 QPS 下，服务系统上的计算负载也会变化。Niyama 受益于动态



(a) Capacity to serve a load of 50 QPS



(b) Maximum goodput in a shared cluster

Figure 7. Capacity evaluation at uniform load

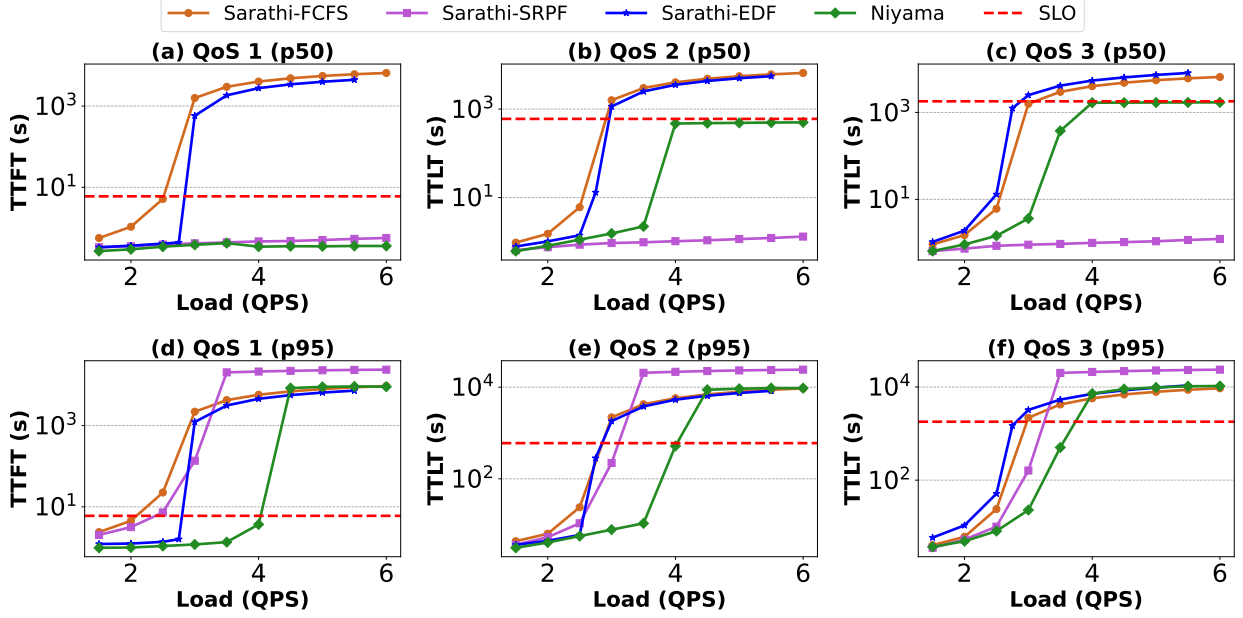


Figure 8. Latency of requests across the three QoS buckets as we vary load in the system

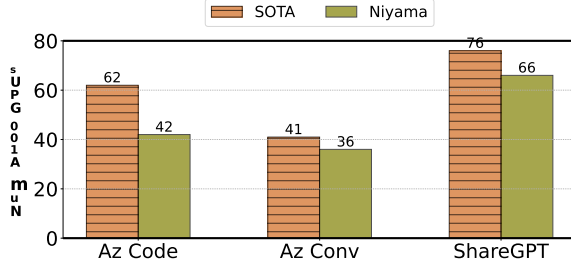
increasing chunk size by exploiting any deadline slack of the non-interactive QoS requests as well as any slack of interactive requests during lower load. On the other hand, the siloed replicas serving the strict QoS requests are limited by the small chunk sizes required to meet the TBT constraints resulting in lower efficiency.

4.1.2 Goodput. We measure the system’s goodput, which we define as the number of requests served per second while meeting the latency targets (p99). We allow at most 1% of total requests to violate their deadlines. Figure 7b shows the goodput while serving requests over a 4-hour period from the Azure-Code dataset on a shared A100 cluster. As shown, NİYAMA achieves 1.5× to 2.4× higher goodput compared to Sarathi-FCFS and 20–40% higher goodput than Sarathi-EDF. These performance benefits stem from a combination of dynamic chunking, hybrid prioritization, and eager relegation. (§4.4) examines the contribution of each of these techniques.

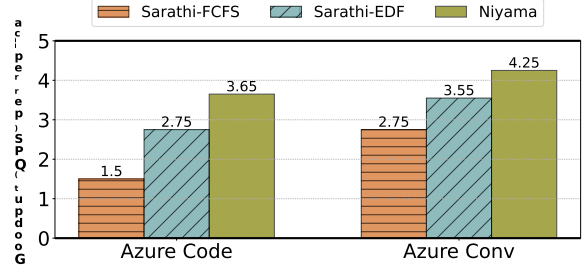
4.2 Latency and SLO violations under Overload

We comprehensively evaluate system behavior at various loads by comparing NİYAMA against baselines on a shared cluster. We measure three key parameters: (1) median and p95 latency (TTFT, TBT, and TTLT) across all requests, (2) percentage of deadline violations across all SLO buckets, and (3) deadline violations in each SLO bucket and violations categorized by request length to assess scheduling fairness. For this evaluation, we add another baseline, Sarathi-SRPF which prioritizes jobs with the lowest pending prefill tokens.

Latency. Figure 8 shows the median and p95 latency across all requests for Llama3-8B on the Azure-Code dataset. As load exceeds the optimal operating point, queuing delay increases because the system cannot process requests as fast as they arrive. This causes a sharp increase in latency for all requests. While this happens in every system, the point where scheduling delay becomes unreasonably large defines



(a) Capacity to serve a load of 50 QPS



(b) Maximum goodput in a shared cluster

图 7. 均匀负载下的容量评估

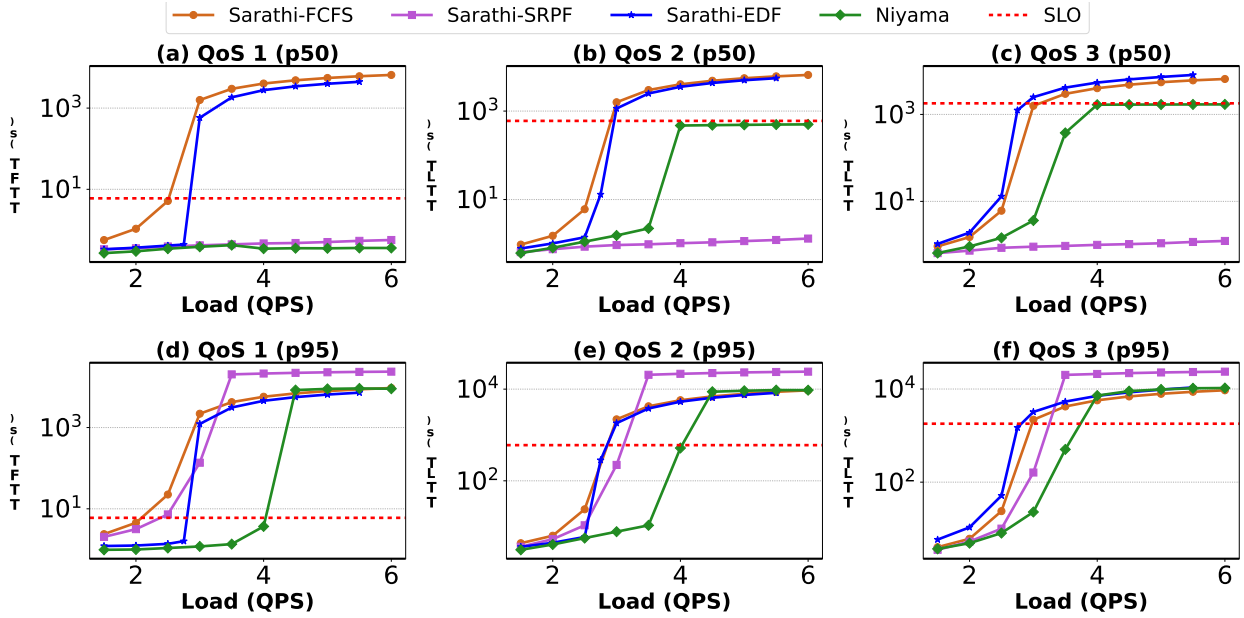


图 8. 当我们改变系统负载时，三个 QoS 桶之间的请求延迟

通过利用非交互式 QoS 请求的任何截止时间裕度以及较低负载期间交互式请求的任何裕度来增加块大小。另一方面，服务于严格 QoS 请求的孤立副本受到满足 TBT 约束所需的小块大小的限制，从而导致效率较低。

4.1.2 有效产出。我们测量系统的吞吐量，将其定义为在满足延迟目标的情况下每秒服务的请求数 (p99)。我们最多允许总请求中的 1% 违反截止日期。图 7b 显示了在 4 小时内从共享 A100 集群上的 Azure-Code 数据集处理请求时的吞吐量。如图所示，与 Sarathi-FCFS 相比，Niyama 的吞吐量高出 1.5× 至 2.4×，比 Sarathi-EDF 高出 20-40%。这些性能优势源于动态分块、混合优先级和急切降级的组合。（§4.4）检查了每种技术的贡献。

4.2 过载下的延迟和 SLO 违规

我们通过将 Niyama 与共享集群上的基线进行比较，全面评估各种负载下的系统行为。我们测量三个关键参数：(1) 所有请求的中值和 p95 延迟 (TTFT、TBT 和 TTL T)，(2) 所有 SLO 存储桶中的截止日期违规百分比，以及 (3) 每个 SLO 存储桶中的截止日期违规以及按请求长度分类的违规，以评估调度公平性。对于此评估，我们添加了另一个基线 Sarathi-SRPF，它优先考虑具有最低待处理预填充令牌的作业。

延迟。图 8 显示了 Azure-Code 数据集上 Llama3-8B 的所有请求的中值和 p95 延迟。当负载超过最佳工作点时，排队延迟会增加，因为系统无法在请求到达时尽快处理请求。这会导致所有请求的延迟急剧增加。虽然这种情况在每个系统中都会发生，但调度延迟变得不合理的大点定义了

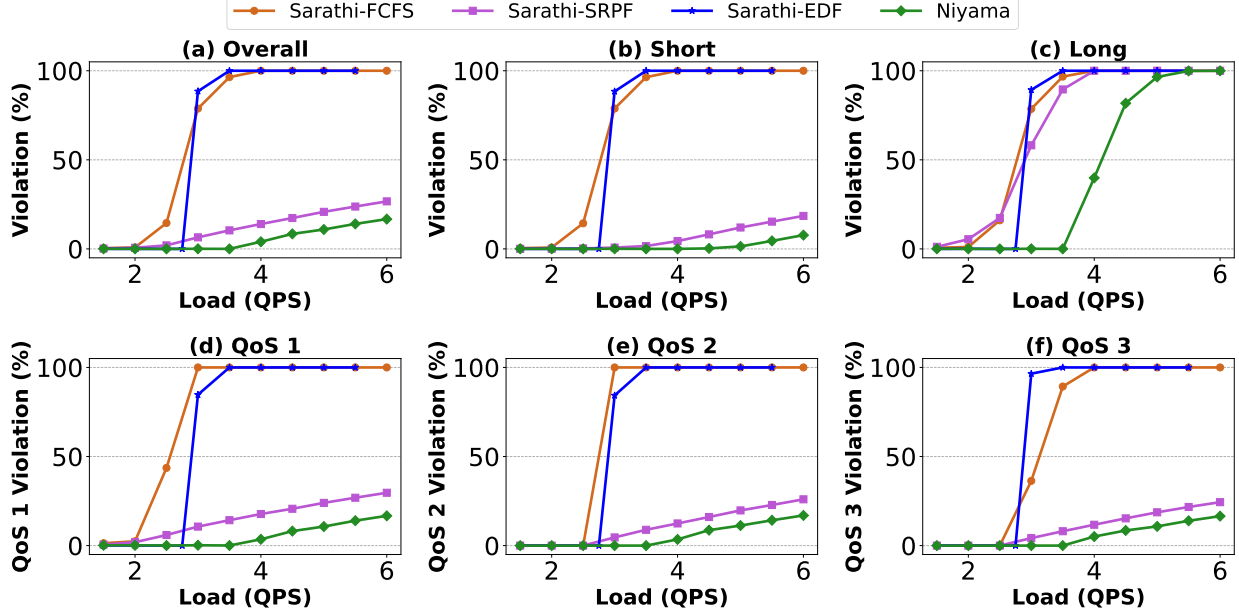


Figure 9. Deadline violations across all jobs, split by request length and QoS buckets

the maximum serviceable load. We skip showing TBT in the plots because across all schemes, the average TBT violations was less than 0.1%, by virtue of carefully chosen chunk size. There are several takeaways from these graphs.

- The state-of-the-art Sarathi-FCFS scheduler ignores individual request deadlines. As load increases, SLOs for jobs with stricter QoS requirements are violated. At heavy overloads, head-of-line blocking causes denial of service to all requests.
- Adding deadline awareness through mechanisms like EDF (Sarathi-EDF) better maintains QoS than Sarathi-FCFS, but doesn't scale well with load because we must sacrifice throughput to meet SLOs for the strictest QoS tier. It also degenerates at high loads similar to FCFS due to head-of-line blocking.
- Schedulers that prioritize short jobs like Sarathi-SRPF maintain good median latency at the expense of tail latency. The p95 latency, however, grows unbounded because SRPF ignores longer requests. Since it is not deadline-aware it prioritizes minimizing latency across requests of all SLO-buckets, which could have otherwise been used to prioritize those with stringent SLOs.
- NIYAMA handles up to 40% higher load while meeting tail latency SLOs in each QoS bucket compared to baselines. Notably, NIYAMA's hybrid prioritization smoothly balances between deadline-prioritizing schemes (EDF) and length-prioritizing ones (SRPF), achieving low median latency without drastically increasing tail latency.

Deadline violations. For the same workload, Figure 9(a) shows the overall percentage of SLO violations across all

requests as load varies. NIYAMA maintains zero deadline violations for up to 30% higher load than the next-best scheme, Sarathi-EDF. Even at extreme overloads, NIYAMA has the fewest deadline violations compared to all other shared-cluster scheduling policies. These lower deadline violation result in the higher goodput we saw in (§4.1.2).

Finally, we analyze whether deadline violations are distributed fairly across request lengths and different QoS buckets. Figure 9(b,c) plot the deadline violations by request length (combined across all QoS buckets). We classify requests as long if their prompt token count is greater than or equal to the 90th percentile, and short otherwise. Our analysis reveals three key patterns:

- Sarathi-FCFS and Sarathi-EDF violate SLOs for short and long requests at similar rates. These schedulers do not differentiate between request lengths. At high loads when head-of-line blocking occurs, all requests violate SLOs due to a cascade effect.
- Sarathi-SRPF shows a very high ratio of violations for long versus short jobs, and ignores all long requests beyond certain load. Even at very low loads (<2 QPS), when other schedulers have no deadline violations, Sarathi-SRPF unnecessarily deprioritizes long requests and misses their deadlines. This approach is not only unfair but also counterproductive in real-world settings where request importance doesn't correlate with length.

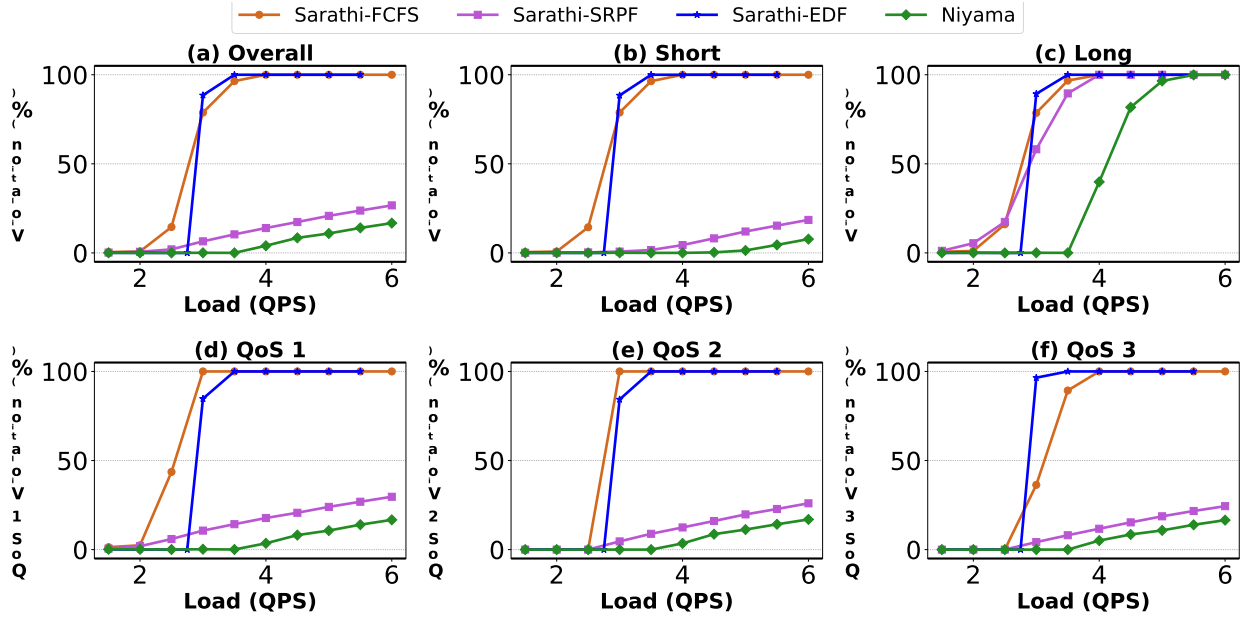


图 9. 所有作业的截止日期违规情况（按请求长度和 QoS 桶划分）

最大可用负载。我们跳过在图中显示 TBT，因为在所有方案中，由于仔细选择了块大小，平均 TBT 违规率低于 0.1%。从这些图表中可以得出一些结论。

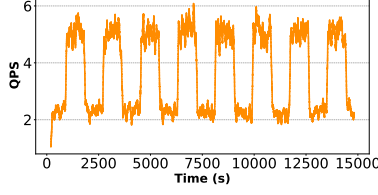
- 最先进的 Sarathi-FCFS 调度程序会忽略单个请求的最后期限。随着负载的增加，具有更严格 QoS 要求的作业的 SLO 就会被违反。在严重过载时，队头阻塞会导致拒绝所有请求的服务。
- 通过 EDF (Sarathi-EDF) 等机制添加截止时间感知可以比 Sarathi-FCFS 更好地维护 QoS，但不能很好地随负载扩展，因为我们必须牺牲吞吐量来满足最严格 QoS 层的 SLO。由于队头阻塞，它在高负载下也会退化，类似于 FCFS。
- 像 Sarathi-SRPF 这样优先考虑短作业的调度程序可以以尾部延迟为代价维持良好的中值延迟。然而，p95 延迟会无限增长，因为 SRPF 会忽略较长的请求。由于它不具有截止日期意识，因此它会优先考虑最小化所有 SLO 存储桶请求中的延迟，否则这些延迟可能会被用来优先考虑那些具有严格 SLO 的请求。
- 与基线相比，Niyama 可处理高出 40% 的负载，同时满足每个 QoS 桶中的尾部延迟 SLO。值得注意的是，Niyama 的混合优先级在截止日期优先方案（EDF）和长度优先方案（SRPF）之间平稳平衡，实现低中值延迟，而不会大幅增加尾部延迟。

违反截止日期。对于相同的工作负载，图 9(a) 显示了所有 SLO 违规的总体百分比

负载变化时的请求。Niyama 保持零期限违规，负载比次优方案 Sarathi-EDF 高出 30%。即使在极端过载的情况下，与其他共享集群调度策略相比，Niyama 的截止日期违规情况也是最少的。这些较低的最后期限违规导致了我们在 (§4.1.2) 中看到的更高的产出。

最后，我们分析截止日期违规是否在请求长度和不同的 QoS 桶中公平分布。图 9(b,c) 按请求长度绘制了截止日期违规情况（所有 QoS 桶的组合）。如果提示令牌计数大于或等于第 90 个百分位数，我们将请求分类为长请求，否则将请求分类为短请求。我们的分析揭示了三个关键模式：

- Sarathi-FCFS 和 Sarathi-EDF 以相似的速率违反短期和长期请求的 SLO。这些调度程序不区分请求长度。在高负载下，当发生队头阻塞时，由于级联效应，所有请求都会违反 SLO。
- Sarathi-SRPF 显示长作业与短作业的违规比例非常高，并且忽略超出特定负载的所有长请求。即使在非常低的负载 (<2 QPS) 下，当其他调度程序没有违反截止日期时，Sarathi-SRPF 也会不必要地降低长请求的优先级并错过截止日期。这种方法不仅不公平，而且在请求重要性与长度不相关的现实环境中还会适得其反。



Scheme	Violations (%)				
	Overall	Important	QoS 0	QoS 1	QoS 2
Sarathi-FCFS	81.88	81.96	97.13	89.14	59.57
Sarathi-EDF	84.12	84.09	79.3	83.27	89.77
NIYAMA	8.64	0	16.03	9.98	0

Figure 10. Workload with varying QPS and overall deadline violations across different schemes

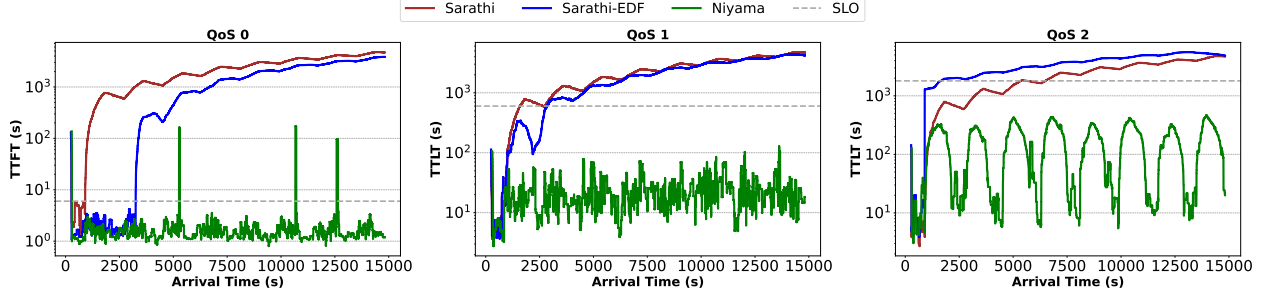


Figure 11. Rolling average of p99 latency of all high-priority requests during a dynamic workload with varying request rates

- NIYAMA achieves balance between these extremes. It does not deprioritize long requests under normal conditions. But during overload, it adjusts the α parameter (§3.1) to incorporate SRPF-like behavior. This approach allows NIYAMA to maintain fairness at reasonable loads while gracefully degrading service as load increases.

Figure 9d-f plots the split of deadline violations across the three constituent QoS buckets. We observe that Sarathi-FCFS first violates requests in the strictest QoS bucket and then continues to the less strict buckets. This happens because Sarathi-FCFS is deadline unaware, and due to head-of-line blocking, it violates requests with the shortest deadlines when they get blocked by other requests. Sarathi-EDF equally misses deadlines across all tiers because it treats all requests equally with respect to their individual deadlines. Sarathi-SRPF shows a pattern similar to Sarathi-FCFS, violating the strictest tier first due to being deadline unaware. However, it has fewer overall violations by ignoring long jobs, which frees up capacity for the larger proportion of short requests. On the contrary, NIYAMA combines the best of these strategies via hybrid prioritization, and achieves fewer overall violations than even Sarathi-SRPF.

4.3 Latency and SLO violations under Transient Overloads

We evaluate whether NIYAMA can gracefully degrade service during transient overload by running an end-to-end evaluation with diurnal load patterns. Load in the system varies dynamically between low (QPS:2.0) and high (QPS:6) points every 15 minutes over a total of 4 hours as shown in Figure 10(a). To evaluate NIYAMA handling of requests

with multiple priorities, we mark a random set of 20% of requests in each QoS bucket as low priority, based on application hints. The remaining 80% of requests in each bucket are marked as high priority or Important.

Figure 10(b) shows the overall deadline violations observed in the system. While the baselines collapse under this load and violate deadlines for all requests, NIYAMA misses deadlines for no important tasks and only 8.75% of all requests. This improvement comes from leveraging application hints to perform eager relegation. Additionally, the throughput gains from dynamic chunking and hybrid prioritization help NIYAMA sustain higher loads.

Figure 11 plots the rolling p99 latency (over 60s windows) of all requests in the system for the three QoS buckets. We see that the baseline Sarathi-FCFS crumbles when the first request burst hits. It cannot recover from the queueing delay and enters request denial mode beyond that point for all classes. While Sarathi-EDF sustains the first burst and absorbs some of it until the second peak, it succumbs to queueing delay beyond this point. NIYAMA elegantly handles both high and low load periods, meeting the p99 latency SLOs for a large majority of requests (all important requests and 92% of all requests). This demonstrates graceful service degradation—proactively dropping a few requests during overload to maintain service levels for the majority, thereby eliminating cascade effects.

4.4 Ablation Studies

We now examine how each component of our system design affects throughput and SLO violations. For this analysis, we tag all requests as important and evaluate three design

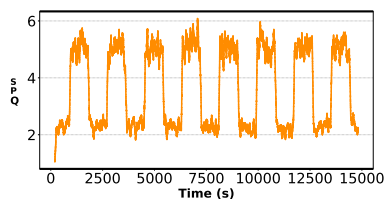


图 10. 不同方案中具有不同 QPS 和总体截止期限违规情况的工作负载

Scheme	Violations (%)				
	Overall	Important	QoS 0	QoS 1	QoS 2
Sarathi-FCFS	81.88	81.96	97.13	89.14	59.57
Sarathi-EDF	84.12	84.09	79.3	83.27	89.77
Niyama	8.64	0	16.03	9.98	0

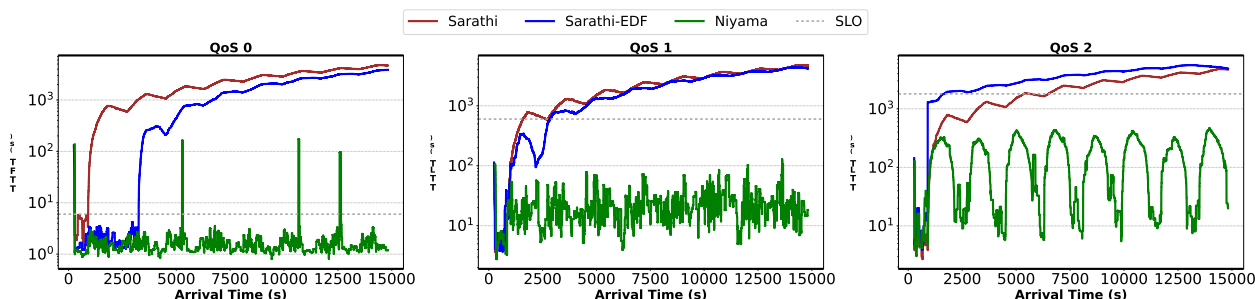


图 11. 在具有不同请求 r_a 的动态工作负载期间，所有高优先级请求的 p99 延迟的滚动平均值

特斯

- 尼亚玛在这些极端之间取得了平衡。在正常情况下，它不会降低长请求的优先级。但在过载期间，它会调整 α 参数（第 3.1 节）以合并类似 SRPF 的行为。这种方法允许 Niyama 在合理的负载下保持公平性，同时随着负载的增加优雅地降低服务质量。

图 9d-f 绘制了三个组成的 QoS 桶中截止期限违规的分布情况。我们观察到 Sarathi-FCFS 首先违反最严格 QoS 桶中的请求，然后继续违反不太严格的桶。发生这种情况是因为 Sarathi-FCFS 不知道截止日期，并且由于队头阻塞，当请求被其他请求阻塞时，它会违反截止日期最短的请求。Sarathi-EDF 在所有级别都会同样错过最后期限，因为它在各自的最后期限方面平等地对待所有请求。Sarathi-SRPF 显示了与 Sarathi-FCFS 类似的模式，由于不知道截止日期而首先违反了最严格的层。然而，它通过忽略长作业而减少了总体违规行为，从而为更大比例的短请求释放了容量。相反，Niyama 通过混合优先级结合了这些策略中的最佳策略，并且总体违规次数甚至比 Sarathi-SRPF 还要少。

对于多个优先级，我们根据应用程序提示将每个 QoS 桶中 20% 的随机请求标记为低优先级。每个存储桶中剩余 80% 的请求被标记为高优先级或重要。

图 10(b) 显示了系统中观察到的总体截止日期违规情况。虽然基线在此负载下崩溃并违反了所有请求的最后期限，但 Niyama 没有错过任何重要任务的最后期限，并且只错过了所有请求的 8.75%。这种改进来自于利用应用程序提示来执行急切的降级。此外，动态分块和混合优先级带来的吞吐量增益有助于 Niyama 维持更高的负载。

图 11 绘制了系统中三个 QoS 桶的所有请求的滚动 p99 延迟（超过 60 秒窗口）。我们看到，当第一个请求爆发时，基线 Sarathi-FCFS 崩溃了。它无法从排队延迟中恢复，并会进入所有类的请求拒绝模式。虽然 Sarathi-EDF 维持第一个突发并吸收其中的一些直到第二个峰值，但超过该点它就会屈服于排队延迟。Niyama 可以优雅地处理高负载和低负载时段，满足大多数请求（所有重要请求和所有请求的 92%）的 p99 延迟 SLO。这展示了优雅的服务降级——在过载期间主动丢弃一些请求以维持大多数的服务水平，从而消除级联效应。

4.3 瞬时过载下的延迟和 SLO 违规

我们通过使用昼夜负载模式运行端到端评估来评估 Niyama 是否可以在瞬时过载期间优雅地降低服务质量。系统负载在总共 4 小时内每 15 分钟在低点 (QPS:2.0) 和高点 (QPS:6) 之间动态变化，如图 10(a) 所示。评估 Niyama 对请求的处理

4.4 消融研究

我们现在检查系统设计的每个组件如何影响吞吐量和 SLO 违规。对于此分析，我们将所有请求标记为重要并评估三个设计

Config	Optimal Load		High load (QPS=6)	
	QPS	% gain	% viol	% impr.
Sarathi-EDF	2.75	-	100	-
NIYAMA (DC)	3.3	20%	74	26%
NIYAMA (DC+ER)	3.6	9%	26	68%
NIYAMA (DC+ER+HP)	3.65	1.4%	16	32%

Table 3. Impact of NIYAMA’s optimizations. (DC:Dynamic Chunking, ER:Eager Relegation, HP:Hybrid Prioritization)

elements—dynamic chunking, hybrid prioritization, and eager relegation—starting with the Sarathi-EDF baseline. Table 3 shows that dynamic chunking provides a 20% boost in throughput, while eager relegation adds another 9%. The impact of hybrid prioritization appears marginal in the optimal load scenario but becomes significant at high load.

To further illustrate the impact of hybrid prioritization, Figure 12 plots the median latency and percentage of deadline violations as we vary system load across three different values of α , our hybrid prioritization parameter. As α increases, the system increasingly deprioritizes longer requests. This significantly reduces median latency for all requests but comes at the cost of violating deadlines for most long requests. This demonstrates the importance of tuning this parameter as load increases to strike a balance between low median latency and fair service for long requests.

5 Related work

We now place our work in the context of relevant literature.

LLM Inference Optimization. Recent systems research has focused extensively on optimizing LLM inference to reduce latency and increase throughput. Orca [26] developed iteration level batching improving the throughput of LLM Inference. vLLM [14] introduced the concept of PagedAttention to manage the KV cache effectively, reducing memory fragmentation and enabling efficient memory sharing across multiple requests. Similarly, Sarathi [4] proposed chunked-prefills and stall-free batching to address the throughput-latency tradeoff. While these advancements have significantly enhanced inference efficiency, they primarily target throughput maximization without explicitly accommodating diverse QoS requirements. Our work builds upon these foundational optimizations to further maximize QoS attainment across multiple service classes.

Multi-Tenant Serving Systems. Multi-tenant serving systems for machine learning models have received significant attention in recent years. Clockwork [12] introduced SLO-aware scheduling for DNN inference assuming a single SLO across all requests, while INFaaS [18] provides an interface for model selection based on accuracy-latency tradeoffs by switching between models. However, these systems don’t address the challenge of co-scheduling requests across different QoS classes on a shared infrastructure and the unique challenges that LLM inference brings therewith.

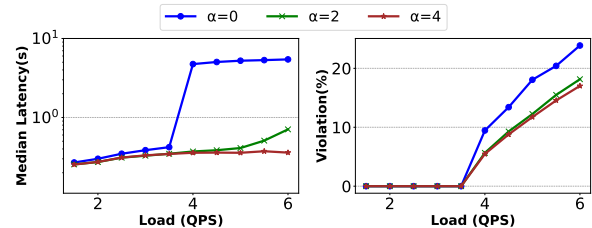


Figure 12. Varying the hybrid prioritization parameter

QoS-Aware Scheduling. Traditional QoS-aware scheduling has been extensively studied in real-time systems literature. Approaches like Earliest Deadline First (EDF) [16] and Rate Monotonic Scheduling [15] provide theoretical foundations for task scheduling with deadlines. In cloud computing, prior work [10, 11] addresses QoS-aware resource management in datacenters by mapping workloads to appropriate resources while minimizing interference. Inspired by the importance of QoS, our work examines the challenges and opportunities in a new domain—LLM serving, with its unique two-phase execution model, high preemption costs, and specialized requirements for handling overload conditions.

Graceful service degradation. Online systems manage overload through complementary techniques including load balancing [6, 9], per-client quotas, client side throttling [21], resource scheduling [7, 19], and graceful degradation [1, 8]. These approaches have been extensively studied in web services, databases, and distributed systems. For example, Bouncer [25] introduces an admission control framework for low-latency data systems that makes intelligent decisions about query admission based on response time objectives. To the best of our knowledge, our work is the first to explore these challenges specifically in the context of LLM inference by leveraging its unique characteristics and identifying novel opportunities for maintaining QoS guarantees during overload conditions.

6 Conclusion

We address the challenge of co-scheduling multiple QoS classes in LLM inference serving, and graceful service degradation during overload. We achieve this using three key techniques: (1) dynamic chunking to opportunistically maximize throughput while meeting latency targets, (2) hybrid prioritization to strike a balance between maintaining low median latency and fairness in serving longer requests, and (3) eager relegation to enable graceful service degradation. Our evaluation across diverse workloads shows that NIYAMA significantly improves QoS attainment compared to state-of-the-art LLM serving systems, particularly under high load. As LLMs power more applications with varying performance needs, we believe that techniques supporting multiple QoS classes will become essential for production deployments.

Config	Optimal Load		High load (QPS=6)	
	QPS	% gain	% viol	% impr.
Sarathi-EDF	2.75	-	100	-
Niyama (DC)	3.3	20%	74	26%
Niyama (DC+ER)	3.6	9%	26	68%
Niyama (DC+ER+HP)	3.65	1.4%	16	32%

表 3. Niyama 优化的影响。(DC: 动态分块、ER: 急切降级、HP: 混合优先级)

元素——动态分块、混合优先级和急切降级——从 Sarathi-EDF 基线开始。表 3 显示，动态分块使吞吐量提高了 20%，而急切降级则又提高了 9%。混合优先级的影响在最佳负载场景中显得微不足道，但在高负载情况下变得显著。

为了进一步说明混合优先级的影响，图 12 绘制了当我们在三个不同的 α （我们的混合优先级参数）值之间改变系统负载时，中值延迟和违反截止日期的百分比。随着 α 的增加，系统越来越不优先考虑较长的请求。这显着减少了所有请求的中值延迟，但代价是违反了大多数长请求的截止日期。这证明了随着负载增加调整此参数的重要性，以在低中值延迟和长请求的公平服务之间取得平衡。

5 相关工作

我们现在将我们的工作置于相关文献的背景下。

LLM 推理优化。最近的系统研究广泛关注优化 LLM 推理以减少延迟并提高吞吐量。Orca [26] 开发了迭代级批处理，提高了 LLM 推理的吞吐量。vLLM [14] 引入了 PagedAttention 的概念来有效管理 KV 缓存，减少内存碎片并实现跨多个请求的高效内存共享。类似地，Sarathi [4] 提出了分块预填充和无停顿批处理来解决吞吐量与延迟之间的权衡。虽然这些进步显着提高了推理效率，但它们主要目标是吞吐量最大化，而没有明确满足不同的 QoS 要求。我们的工作建立在这些基础优化的基础上，以进一步最大化跨多个服务类别的 QoS 实现。

多租户服务系统。近年来，机器学习模型的多租户服务系统受到了广泛关注。Clockwork [12] 为 DNN 推理引入了 SLO 感知调度，假设所有请求都采用单个 SLO，而 INFaaS [18] 通过在模型之间切换，提供了基于精度-延迟权衡的模型选择接口。然而，这些系统并没有解决在共享基础设施上跨不同 QoS 类别共同调度请求的挑战以及 LLM 推理带来的独特挑战。

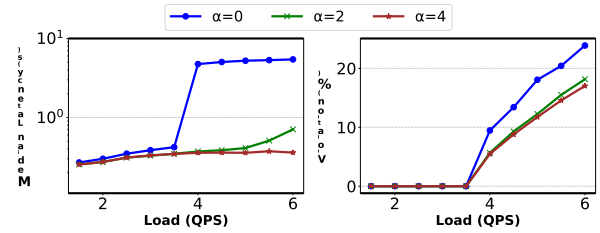


图 12. 改变混合优先级参数

QoS 感知调度。传统的 QoS 感知调度已在实时系统文献中得到广泛研究。最早截止日期优先 (EDF) [16] 和速率单调调度 [15] 等方法为具有截止日期的任务调度提供了理论基础。在云计算中，先前的工作 [10, 11] 通过将工作负载映射到适当的资源同时最大限度地减少干扰来解决数据中心中的 QoS 感知资源管理问题。受 QoS 重要性的启发，我们的工作研究了新领域 (LLM 服务) 中的挑战和机遇，该领域具有独特的两阶段执行模型、高抢占成本以及处理过载条件的特殊要求。

优雅的服务降级。在线系统通过补充技术来管理过载，包括负载均衡 [6, 9]、每个客户端配额、客户端限制 [21]、资源调度 [7, 19] 和优雅降级 [1, 8]。这些方法已在 Web 服务、数据库和分布式系统中得到广泛研究。例如，Bouncer [25] 为低延迟数据系统引入了准入控制框架，该框架根据响应时间目标做出有关查询准入的智能决策。据我们所知，我们的工作第一个专门在 LLM 推理背景下探索这些挑战的工作，利用其独特的特征并确定在过载条件下维持 QoS 保证的新机会。

6 结论

我们解决了 LLM 推理服务中共同调度多个 QoS 类别的挑战，以及过载期间服务的平稳降级。我们使用三种关键技术来实现这一目标：（1）动态分块，在满足延迟目标的同时，机会性地最大化吞吐量；（2）混合优先级，在保持低中值延迟和服务较长请求的公平性之间取得平衡；（3）急切降级，以实现优雅的服务降级。我们对不同工作负载的评估表明，与最先进的 LLM 服务系统相比，Niyama 显着提高了 QoS 水平，尤其是在高负载下。随着 LLM 为更多具有不同性能需求的应用程序提供支持，我们相信支持多个 QoS 类别的技术对于生产部署将变得至关重要。

References

- [1] Sameer Agarwal, Barzan Mozafari, Aurojit Panda, Henry Milner, Samuel Madden, and Ion Stoica. 2013. BlinkDB: queries with bounded errors and bounded response times on very large data. In *Eighth Eurosys Conference 2013, EuroSys '13, Prague, Czech Republic, April 14-17, 2013*, Zdenek Hanzálek, Hermann Härtig, Miguel Castro, and M. Frans Kaashoek (Eds.). ACM, 29–42. <https://doi.org/10.1145/2465351.2465355>
- [2] Amey Agrawal, Anmol Agarwal, Nitin Kedia, Jayashree Mohan, Souvik Kundu, Nipun Kwatra, Ramachandran Ramjee, and Alexey Tumanov. 2024. Etalon: Holistic Performance Evaluation Framework for LLM Inference Systems. *arXiv e-prints*, Article arXiv:2407.07000 (July 2024), arXiv:2407.07000 pages. <https://doi.org/10.48550/arXiv.2407.07000> [cs.LG]
- [3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework For LLM Inference. *Proceedings of The Seventh Annual Conference on Machine Learning and Systems, 2024, Santa Clara* (2024).
- [4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [5] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 [cs.LG]
- [6] David Chou, Tianyin Xu, Kaushik Veeraraghavan, Andrew Newell, Sonia Margulis, Lin Xiao, Pol Mauri Ruiz, Justin Meza, Kiryong Ha, Shruti Padmanabha, Kevin Cole, and Dmitri Perelman. 2019. Taiji: managing global user traffic for large-scale internet services at the edge. *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (2019). <https://api.semanticscholar.org/CorpusID:204812238>
- [7] M. Colajanni, P.S. Yu, and D.M. Dias. 1997. Scheduling algorithms for distributed Web servers. In *Proceedings of 17th International Conference on Distributed Computing Systems*. 169–176. <https://doi.org/10.1109/ICDCS.1997.598025>
- [8] Alejandro Forero Cuervo. 2017. Handling overload. Site Reliability Engineering: How Google runs production systems. (2017).
- [9] Alejandro Forero Cuervo. 2017. Load balancing in the datacenter. Site Reliability Engineering: How Google runs production systems. (2017).
- [10] Christina Delimitrou and Christos Kozyrakis. 2013. Paragon: QoS-aware scheduling for heterogeneous datacenters. In *Architectural Support for Programming Languages and Operating Systems, ASPLOS 2013, Houston, TX, USA, March 16-20, 2013*, Vivek Sarkar and Rastislav Bodík (Eds.). ACM, 77–88. <https://doi.org/10.1145/2451116.2451125>
- [11] Christina Delimitrou and Christos Kozyrakis. 2014. Quasar: resource-efficient and QoS-aware cluster management. In *Architectural Support for Programming Languages and Operating Systems, ASPLOS 2014, Salt Lake City, UT, USA, March 1-5, 2014*, Rajeev Balasubramanian, Al Davis, and Sarita V. Adve (Eds.). ACM, 127–144. <https://doi.org/10.1145/2541940.2541941>
- [12] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving {DNNs} like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 443–462.
- [13] Shashwat Jaiswal, Kunal Jain, Yogesh Simmhan, Anjaly Parayil, Ankur Mallick, Rujia Wang, Renee St Amant, Chetan Bansal, Victor Rühle, Anoop Kulkarni, et al. 2025. Serving Models, Fast and Slow: Optimizing Heterogeneous LLM Inferencing Workloads at Scale. *arXiv preprint arXiv:2502.14617* (2025).
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention (SOSP '23). Association for Computing Machinery, New York, NY, USA, 611–626. <https://doi.org/10.1145/3600006.3613165>
- [15] John P. Lehoczky, Lui Sha, and Y. Ding. 1989. The Rate Monotonic Scheduling Algorithm: Exact Characterization and Average Case Behavior. In *Proceedings of the Real-Time Systems Symposium - 1989, Santa Monica, California, USA, December 1989*. IEEE Computer Society, 166–171. <https://doi.org/10.1109/REAL.1989.63567>
- [16] C. L. Liu and James W. Layland. 1973. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *J. ACM* 20, 1 (1973), 46–61. <https://doi.org/10.1145/321738.321743>
- [17] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. arXiv:2405.04437 [cs.LG] <https://arxiv.org/abs/2405.04437>
- [18] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. 2021. {INFaaS}: Automated Model-less Inference Serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 397–411.
- [19] Bianca Schroeder and Mor Harchol-Balter. 2006. Web servers under overload: How scheduling can help. *ACM Trans. Internet Technol.* 6, 1 (Feb. 2006), 20–52. <https://doi.org/10.1145/1125274.1125276>
- [20] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2024. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. arXiv:2408.00741 [cs.AI] <https://arxiv.org/abs/2408.00741>
- [21] Mike Ulrich. 2017. Addressing cascading failures. Site Reliability Engineering: How Google runs production systems. (2017).
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [23] vLLM Project. 2024. [RFC] Upstream Chunked Prefill #3130. <https://github.com/vllm-project/vllm/issues/3130>.
- [24] Guan Wang, Sijie Cheng, Xianyu Zhan, Xiangang Li, Sen Song, and Yang Liu. 2024. OpenChat: Advancing Open-source Language Models with Mixed-Quality Data. arXiv:2309.11235 [cs.CL] <https://arxiv.org/abs/2309.11235>
- [25] Hao Xu and Juan A. Colmenares. 2023. Admission Control with Response Time Objectives for Low-latency Online Data Systems. arXiv:2312.15123 [cs.DB] <https://arxiv.org/abs/2312.15123>
- [26] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [27] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] <https://arxiv.org/abs/2312.07104>

参考

- [1] Sameer Agarwal, Barzan Mozafari, Aurojit Panda, Henry Milner, Samuel Madden 和 Ion Stoica. 2013. BlinkDB: 对非常大的数据进行有限错误和有限响应时间的查询。2013 年第八届 Eurosys 会议, Euro Sys '13, 捷克共和国布拉格, 2013 年 4 月 14-17 日, Zdenek Hanzálek、Hermann Härtig、Miguel Castro 和 M. Frans Kaashoek (编辑)。ACM, 29-42. <https://doi.org/10.1145/2465351.2465355> [2] Amey Agrawal、Anmol Agarwal、Nitin Kedia、Jayashree Mohan、Souvik Kundu、Nipun Kwatra、Ramachandran Ramjee 和 Alexey Tumanov. 2024. Etalon: 大语言模型系统的整体性能评估框架。arXiv 电子印刷品, 文章 arXiv:2407.07000 (2024 年 7 月), arXiv:2407.07000 页。 <https://doi.org/10.48550/arXiv.2407.07000> [cs.LG] [3] Amey Agrawal、Nitin Kedia、Jayashree Mohan、Ashish Panwar、Nipun Kwatra、Bhargav S Gulavani、Ramachandran Ramjee 和 Alexey Tumanov. 2024. Vidur: LLM 推理的大规模模拟框架。第七届机器学习和系统年会论文集, 2024 年, 圣克拉拉 (2024 年)。 [4] Amey Agrawal、Nitin Kedia、Ashish Panwar、Jayashree Mohan、Nipun Kwatra、Bhargav Gulavani、Alexey Tumanov 和 Ramachandran Ramjee. 2024 年。通过 {Sarithi-Serve} 克服 {LLM} 推理中的 {吞吐量与延迟} 之间的权衡。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。117-134。 [5] Amey Agrawal、Ashish Panwar、Jayashree Mohan、Nipun Kwatra、Bhargav S. Gulavani 和 Ramachandran Ramjee. 2023. SARATHI: 通过分块预填充的搭载解码进行高效的 LLM 推理。arXiv:2308.16369 [cs.LG] [6] David Chou、Tianyin Xu、Kaushik Veeraraghavan、Andrew Newell、Sonia Margulis、Lin Xiao、Pol Mauri Ruiz、Justin Meza、Kiryong Ha、Shruti Padmanabha、Kevin Cole 和 Dmitri Perelman. 2019. Taiji: 管理边缘大规模互联网服务的全球用户流量。第 27 届 ACM 操作系统原理研讨会论文集 (2019 年)。 <https://api.semanticscholar.org/CorpusID:204812238> [7] M. Colajanni, P.S. 于和 D.M. 迪亚斯. 1997. 分布式 Web 服务器的调度算法。第 17 届国际分布式计算系统会议论文集。169-176。 <https://doi.org/10.1109/ICDCS.1997.598025> [8] 亚历杭德罗·弗雷罗·库尔沃. 2017. 处理超载。站点可靠性工程: Google 如何运行生产系统。(2017)。 [9] 亚历杭德罗·弗雷罗·库尔沃. 2017. 数据中心的负载平衡。站点可靠性工程: Google 如何运行生产系统。(2017)。 [10] 克里斯蒂娜·德利米特鲁和克里斯托·科兹拉基斯. 2013 年。Paragon: 异构数据中心的 QoS 感知调度。编程语言和操作系统的架构支持, ASPLOS 2013, 美国德克萨斯州休斯顿, 2013 年 3 月 16-20 日, Vivek Sarkar 和 Rastislav Bodík (编辑)。ACM, 77-88. <https://doi.org/10.1145/2451116.2451125> [11] Christina Delimitrou 和 Christos Kozyrakis. 2014. Quasar: 资源高效且 QoS 感知的集群管理。在编程语言和操作系统的架构支持中, ASPLOS 2014, 美国犹他州盐湖城, 2014 年 3 月 1-5 日, Rajeev Balasubramanian、Al Davis 和 Sarita V. Adve (编辑)。ACM, 127-144. <https://doi.org/10.1145/2541940.2541941> [12] Arpan Gujarati、Reza Karimi、Safya Alzayat、Weihao、Antoine Kaufmann、Ymir Vigfusson 和 Jonathan Mace. 2020 年。像 Clockwork 一样为 {DNN} 提供服务: 自下而上的性能可预测性。第 14 届 USENIX 操作系统设计与实现研讨会 (OSDI 20)。443-462。 [13] Shashwat Jaiswal、Kunal Jain、Yogesh Simmhan、Anjaly Parayil、Ankur Mallick、Ruji Wang、Renee St Amant、Chetan Bansal、Victor Rühle、Anoop Kulrani 等。2025. 服务模型, 快与慢: 大规模优化异构 LLM 推理工作负载。arXiv 预印本 arXiv:2502.14617 (2025)。
- [14] Woosuk Kwon, Zhuohan Li, 庄思源, Yingheng, Lianmin Cheng, Codyhao Yu, Joseph Gonzalez, Hao Zhang, Ion Stoica. 2023. 使用 PagedAttention 进行大型语言模型的高效内存管理 (SOSP '23)。计算机协会, 美国纽约州纽约市, 611-626. <https://doi.org/10.1145/3600006.3613165> [15] John P. Lehoczky、Lui Sha 和 Y. Ding. 1989. 速率单调调度算法: 精确表征和平均情况行为。实时系统研讨会论文集 - 1989, 美国加利福尼亚州圣莫尼卡, 1989 年 12 月。IEEE 计算机协会, 166-171. <https://doi.org/10.1109/REAL.1989.63567> [16] C. L. Liu 和 James W. Layland. 1973. 硬实时环境中多道程序设计的调度算法。JACM 20, 1 (1973), 46-61. <https://doi.org/10.1145/321738.321743> [17] Ramya Prabhu、Ajay Nayak、Jayashree Mohan、Ramachandran Ramjee 和 Ashish Panwar. 2025. vAttention: 在没有 PagedAttention 的情况下为 LLM 提供动态内存管理。arXiv:2405.04437 [cs.LG] <https://arxiv.org/abs/2405.04437> [18] Francisco Romero、Qian Li、Neeraja J Yadwadkar 和 Christos Kozyrakis. 2021. {INFaaS}: 自动化无模型推理服务。2021 年 USENIX 年度技术会议 (USENIX ATC 21)。397-411。 [19] 比安卡·施罗德和莫尔·哈科尔-巴尔特. 2006 年。网络服务器过载: 调度如何提供帮助。ACM 翻译。互联网技术。6, 1 (2006 年 2 月), 20-52. <https://doi.org/10.1145/1125274.1125276> [20] Jovan Stojkovic、Chaojie 张、Íñigo Goiri、Josep Torrelas 和 Esha Choukse. 2024. DynamoLLM: 设计 LLM 推理集群以提高性能和能源效率。arXiv:2408.00741 [cs.AI] <https://arxiv.org/abs/2408.00741> [21] 迈克·乌里希. 2017 年。解决级联故障。站点可靠性工程: Google 如何运行生产系统。(2017)。 [22] Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Lion Jones、Aidan N Gomez、Łukasz Kaiser 和 Illia Polosukhin. 2017 年。您所需要的就是注意力。《神经信息处理系统进展》, I. Guyon、U. Von Luxburg、S. Bengio、H. Wallach、R. Fergus、S. Vishwanathan 和 R. Garnett (编辑), 卷。30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf [23] vLLM 项目。2024. [RFC] 上游分块预填充#3130。 <https://github.com/vllm-project/vllm/issues/3130>。 [24] 王冠, 程思杰, 詹贤元, 李贤刚, 宋森, 刘阳. 2024. OpenChat: 利用混合质量数据推进开源语言模型。arXiv:2309.11235 [cs.CL] <https://arxiv.org/abs/2309.11235> [25] 徐浩和 Juan A. Colmenares. 2023. 低延迟在线数据系统的具有响应时间目标的准入控制。arXiv:2312.15123 [cs.DB] <https://arxiv.org/abs/2312.15123> [26] Gyeong-In Yu、Joo Seong Jeong、Geon-Woo Kim、Soojeong Kim 和 Byung-Gon Chun. 2022. Orca: 基于 Transformer 的生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22)。USENIX 协会, 卡尔斯巴德, 加利福尼亚州, 521-538. <https://www.usenix.org/conference/osdi22/presentation/yyu> [27] Lianmin Cheng、Liangsheng Yin、Zhiqiang Xie、Chuyue Sun、Jeff Huang、Codyhao Yu、Shiyi Cao、Christos Kozyrakis、Ion Stoica、Joseph E. Gonzalez、Clark Barrett 和 Ying Shen. 2024. SGLang: 结构化语言模型程序的高效执行。arXiv:2312.07104 [cs.AI] <https://arxiv.org/abs/2312.07104>